

Assessment 1

Name: Iliana Peters

Student Number: 35723483

Dataset introduction

The dataset used in this assignment is the Uber Data Analytics Dashboard for the entire 2024 year. It includes the data of various Uber rides, from users and drivers, including dates, locations, timing data, prices, and ratings. A main business case that can be made from this data relates to influential factors in patient satisfaction, leading to the question 'What are the most influential factors that impact rider satisfaction?' Various metrics could be investigated such as ride distance to booking value, pickup duration, vehicle type, and travel duration to ride distance. Additionally, Customer Cancellation Reasons is a free text column that can be textually analysed for trends and additional insights.

Part A: Analytical Query Design and Implementation

Part A assesses your ability to design, implement, and evaluate a non-trivial analytical query using Apache Spark. The focus is not only on producing correct results, but also on demonstrating an understanding of how Spark processes distributed workloads.

1a. The Business Query

Design and implement a non-trivial business query that requires the following operations:

- A window function, for example, running total, rank within a partition, moving average, cumulative statistics
- High-velocity activity spikes: identify users whose transactions count in any single hour exceeds three standard deviations above their personal hourly mean
- A time-based analysis using date or timestamp attributes

Using the above operations and relating them to the Uber dataset, the following queries are created and will be investigated in the following sections:

1. Calculate statistics on Ride Distance, Booking Value, Pickup Duration, Travel Duration and Driver Rating. These should be ranked by Driver Ratings within the partition.

2024-08-23 08:56:10	""CNR8494506""	Completed	""CID920281
6""	Auto	Khandsa	Malviya Nagar
25.8		null	13.4
null		null	null
null	627	13.58	4.9
Debit Card			4.9
2024-10-21 17:17:25	""CNR8906825""	Completed	""CID261091
4""	Premier Sedan	Central Secretariat	Inderlok
28.5		null	13.1
null		null	null
null	416	34.02	4.6
UPI			5.0
2024-09-16 22:08:00	""CNR1950162""	Completed	""CID993354
2""	Bike	Ghitorni Village	Khan Market
19.6		null	5.3
null		null	null
null	737	48.21	4.1
UPI			4.3

```

+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
-----+-----+

```

only showing top 5 rows

root

```

|-- Date: string (nullable = true)
|-- Time: string (nullable = true)
|-- Booking ID: string (nullable = true)
|-- Booking Status: string (nullable = true)
|-- Customer ID: string (nullable = true)
|-- Vehicle Type: string (nullable = true)
|-- Pickup Location: string (nullable = true)
|-- Drop Location: string (nullable = true)
|-- Avg VTAT: string (nullable = true)
|-- Avg CTAT: string (nullable = true)
|-- Cancelled Rides by Customer: string (nullable = true)
|-- Reason for cancelling by Customer: string (nullable = true)
|-- Cancelled Rides by Driver: string (nullable = true)
|-- Driver Cancellation Reason: string (nullable = true)
|-- Incomplete Rides: string (nullable = true)
|-- Incomplete Rides Reason: string (nullable = true)
|-- Booking Value: string (nullable = true)
|-- Ride Distance: string (nullable = true)
|-- Driver Ratings: string (nullable = true)
|-- Customer Rating: string (nullable = true)
|-- Payment Method: string (nullable = true)

```

Out[142... 150000

```

In [143... #convert numerical columns to float from string
from pyspark.sql.functions import col,concat_ws
from pyspark.sql.types import DateType

# Replace string 'null' with true None/null globally

```

```
df = df.replace("null", None)

float_col = ["Avg VTAT", "Avg CTAT", "Ride Distance", "Driver Ratings"]
df = df.select([col(c).cast("float") if c in float_col else col(c)
df = df.withColumn("Datetime", concat_ws(" ", col("Date"), col("Time")
df.printSchema()
df.show(5)
```

root

```
-- Date: string (nullable = true)
-- Time: string (nullable = true)
-- Booking ID: string (nullable = true)
-- Booking Status: string (nullable = true)
-- Customer ID: string (nullable = true)
-- Vehicle Type: string (nullable = true)
-- Pickup Location: string (nullable = true)
-- Drop Location: string (nullable = true)
-- Avg VTAT: float (nullable = true)
-- Avg CTAT: float (nullable = true)
-- Cancelled Rides by Customer: string (nullable = true)
-- Reason for cancelling by Customer: string (nullable = true)
-- Cancelled Rides by Driver: string (nullable = true)
-- Driver Cancellation Reason: string (nullable = true)
-- Incomplete Rides: string (nullable = true)
-- Incomplete Rides Reason: string (nullable = true)
-- Booking Value: float (nullable = true)
-- Ride Distance: float (nullable = true)
-- Driver Ratings: float (nullable = true)
-- Customer Rating: string (nullable = true)
-- Payment Method: string (nullable = true)
-- Datetime: timestamp (nullable = true)
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|      Date|      Time|      Booking ID| Booking Status|      Customer
ID| Vehicle Type|      Pickup Location|      Drop Location|Avg VTAT|Avg
CTAT|Cancelled Rides by Customer|Reason for cancelling by Customer|C
ancelled Rides by Driver|Driver Cancellation Reason|Incomplete Rides
|Incomplete Rides Reason|Booking Value|Ride Distance|Driver Ratings|
Customer Rating|Payment Method|      Datetime|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|2024-03-23|12:29:38|""CNR5884300""|No Driver Found|""CID198211
1""|
eBike|      Palam Vihar|      Jhilmil|      NULL|
NULL|      NULL|      NULL|
NULL|      NULL|      NULL|
NULL|      NULL|      NULL|
NULL|2024-03-23 12:29:38|
```

```

|2024-11-29|18:01:39|""CNR1326809""|      Incomplete|""CID460480
2""|      Go Sedan|      Shastri Nagar|Gurgaon Sector 56|      4.9|
14.0|      NULL|      NULL|      NULL|
NULL|      NULL|      NULL|      1|      Vehicle Break
down|      237.0|      5.73|      NULL|      NULL|
UPI|2024-11-29 18:01:39|
|2024-08-23|08:56:10|""CNR8494506""|      Completed|""CID920281
6""|      Auto|      Khandsa|      Malviya Nagar|      13.4|
25.8|      NULL|      NULL|      NULL|
NULL|      NULL|      NULL|      NULL|
NULL|      627.0|      13.58|      4.9|      4.9|
Debit Card|2024-08-23 08:56:10|
|2024-10-21|17:17:25|""CNR8906825""|      Completed|""CID261091
4""|Premier Sedan|Central Secretariat|      Inderlok|      13.1|
28.5|      NULL|      NULL|      NULL|
NULL|      NULL|      NULL|      NULL|
NULL|      416.0|      34.02|      4.6|      5.0|
UPI|2024-10-21 17:17:25|
|2024-09-16|22:08:00|""CNR1950162""|      Completed|""CID993354
2""|      Bike|      Ghitorni Village|      Khan Market|      5.3|
19.6|      NULL|      NULL|      NULL|
NULL|      NULL|      NULL|      NULL|
NULL|      737.0|      48.21|      4.1|      4.3|
UPI|2024-09-16 22:08:00|
+-----+-----+-----+-----+-----+
-+-----+-----+-----+-----+-----+
---+-----+-----+-----+-----+
-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
-----+-----+-----+-----+
only showing top 5 rows

```

```

In [144... #after confirming the Datetime column matches, the seperate Date an
df = df.drop("Date", "Time")
df.show(5)

```

Booking ID	Booking Status	Customer ID	Vehicle Type	Pickup Location	Drop Location	Avg VTAT	Avg CTAT	Cancelled Rides by Customer	Reason for cancelling by Customer	Cancelled Rides by Driver	Driver Cancellation Reason	Incomplete Rides	Incomplete Rides Reason	Booking Value	Ride Distance	Driver Ratings	Customer Rating	Payment Method	Datetime
""CNR5884300""	No Driver Found	""CID1982111""	eBike	Palam Vihar	Jhilmil	NULL	NULL	NULL		NULL		NULL		NULL		NULL		NULL	2024-03-23 12:29:38
""CNR1326809""	Incomplete	""CID4604802""	Go Sedan	Shastri Nagar	Gurgaon Sector 56	4.9	14.0	NULL		NULL		1	Vehicle Breakdown	237.0		5.73		UPI	2024-11-29 18:01:39
""CNR8494506""	Completed	""CID9202816""	Auto	Khandsa	Malviya Nagar	13.4	25.8	NULL		NULL		NULL		627.0		13.58	4.9	Debit Card	2024-08-23 08:56:10
""CNR8906825""	Completed	""CID2610914""	Premier Sedan	Central Secretariat	Inderlok	13.1	28.5	NULL		NULL		NULL		416.0		34.02	4.6	UPI	2024-10-21 17:17:25
""CNR1950162""	Completed	""CID9933542""	Bike	Ghitorni Village	Khan Market	5.3	19.6	NULL		NULL		NULL		737.0		48.21	4.1	UPI	2024-09-16 22:08:00

2. DataFrame Implementation

Implement the query using the Spark DataFrame API

2a. Dataframe Investigation

Below is the full workings to create the queries in using Dataframes.

Intermediate steps and working notes will be shown

```
In [145... #BUSINESS QUERY 1 - cumulative statistics on the Ride Distance, Boo  
# These should be ranked by Driver Ratings within the partition.  
  
print(df.count())  
#filter out null ratings and store in cache as intermediate dataframe  
filter_1 = df.dropna(subset=["Driver Ratings"]).cache()  
print(filter_1.count())  
filter_1.describe("Driver Ratings").show()
```

150000

93000

Driver Ratings	
count	93000
mean	4.230992466042118
stddev	0.4368714755910542
min	3.0
max	5.0

```
In [146... #add a column that would specify the customer rating range into 0-3
from pyspark.sql.functions import when
filter_1 = filter_1.withColumn("Driver_Rating_Range",when(col("Driver
                                when(col("Driver Ratings")<=3.5, "3-
                                when(col("Driver Ratings")<=4.5, "4-
filter_1.show(20)
```

```
+-----+-----+-----+-----+-----+
|          Booking ID|Booking Status|      Customer ID| Vehicle Type|
Pickup Location|   Drop Location|Avg VTAT|Avg CTAT|Cancelled Rides b
y Customer|Reason for cancelling by Customer|Cancelled Rides by Driv
er|Driver Cancellation Reason|Incomplete Rides|Incomplete Rides Reas
on|Booking Value|Ride Distance|Driver Ratings|Customer Rating|Paymen
t Method|           Datetime|Driver_Rating_Range|
+-----+-----+-----+-----+-----+
```

["CNR8494506"]	Completed	["CID9202816"]	Auto	
Khandsa Malviya Nagar	13.4 25.8			NU
LL	NULL		NULL	
NULL	NULL	NULL	627.0	
13.58	4.9	4.9	Debit Card 2024-08-23 08:5	
6:10	4.5-5			
["CNR8906825"]	Completed	["CID2610914"]	Premier Sedan Cent	
ral Secretariat	Inderlok	13.1 28.5		
NULL		NULL		NULL
NULL	NULL	NULL	416.0	
34.02	4.6	5.0	UPI 2024-10-21 17:1	
7:25	4.5-5			
["CNR1950162"]	Completed	["CID9933542"]	Bike	G
hitorni Village	Khan Market	5.3 19.6		
NULL		NULL		NULL
NULL	NULL	NULL	737.0	
48.21	4.1	4.3	UPI 2024-09-16 22:0	
8:00	4-4.5			
["CNR4096693"]	Completed	["CID4670564"]	Auto	
AIIMS Narsinghpur	5.1 18.1			NULL
	NULL		NULL	
NULL	NULL	NULL	316.0	
4.85	4.1	4.6	UPI 2024-02-06 09:44:	
56	4-4.5			
["CNR2002539"]	Completed	["CID6800553"]	Go Mini	
Vaishali Punjabi Bagh	7.1 20.4			N
ULL	NULL		NULL	
NULL	NULL	NULL	640.0	
41.24	4.0	4.1	UPI 2024-06-17 15:4	
5:58	3.5-4			
["CNR6568000"]	Completed	["CID8610436"]	Auto	
Mayur Vihar Cyber Hub	12.1 16.5			
NULL	NULL		NULL	
NULL	NULL	NULL	136.0	
6.56	4.4	4.2	UPI 2024-03-19 17:37:	
37	4-4.5			
["CNR9070334"]	Completed	["CID6680340"]	Auto	
Udyog Bhawan Dwarka Sector 21	7.7 18.9			
NULL	NULL		NULL	
NULL	NULL	NULL	181.0	
19.84	4.2	4.9	Cash 2024-06-14 16:2	
4:12	4-4.5			
["CNR2987763"]	Completed	["CID2669710"]	Go Mini	
Malviya Nagar Ghitorni Village	12.2 28.2			
NULL	NULL		NULL	
NULL	NULL	NULL	394.0	
21.44	4.1	4.7	UPI 2024-09-11 19:2	
9:39	4-4.5			
["CNR8962232"]	Completed	["CID1789354"]	Go Mini	
Madipur GTB Nagar	14.0 30.9			NU
LL	NULL		NULL	
NULL	NULL	NULL	836.0	
39.55	4.7	4.4	UPI 2024-10-18 18:2	
8:53	4.5-5			
["CNR2390352"]	Completed	["CID5432215"]	Auto	
Jama Masjid Khan Market	8.5 36.9			

NULL		NULL		NULL
NULL	NULL		NULL	410.0
34.76	4.0	4.9	Uber Wallet	2024-06-07 15:0
5:35	3.5-4			
""CNR3221338""	Completed	""CID2581698""	Premier Sedan	
IGI Airport	Madipur	5.6	27.5	
NULL		NULL		NULL
NULL	NULL		NULL	401.0
21.97	4.9	4.3	UPI	2024-07-01 10:5
1:16	4.5-5			
""CNR9843229""	Completed	""CID4929303""	Go Sedan	
Punjabi Bagh	Mansarovar Park	6.1	33.7	
NULL		NULL		NULL
NULL	NULL		NULL	264.0
13.64	3.8	4.7	Uber Wallet	2024-12-14 21:2
7:04	3.5-4			
""CNR7911434""	Completed	""CID8465004""	Auto	
Greater Noida	Botanical Garden	7.0	21.3	
NULL		NULL		NULL
NULL	NULL		NULL	183.0
4.5	5.0	4.2	UPI	2024-07-29 06:28:2
2	4.5-5			
""CNR6178330""	Completed	""CID7719678""	Go Mini	
Tis Hazari	IMT Manesar	10.4	25.1	
NULL		NULL		NULL
NULL	NULL		NULL	713.0
38.03	3.7	3.6	UPI	2024-02-23 11:0
2:45	3.5-4			
""CNR5309981""	Completed	""CID3870048""	Go Mini	
Noida Sector 18	Old Gurgaon	7.9	41.1	
NULL		NULL		NULL
NULL	NULL		NULL	113.0
14.53	4.6	4.9	UPI	2024-03-09 20:1
4:24	4.5-5			
""CNR2978950""	Completed	""CID2977484""	Auto	
Kanhaiya Nagar	Anand Vihar	13.6	25.2	
NULL		NULL		NULL
NULL	NULL		NULL	767.0
7.39	3.8	4.6	Debit Card	2024-08-28 16:30:
14	3.5-4			
""CNR2806943""	Completed	""CID7550481""	Uber XL	
Okhla	Barakhamba Road	5.1	20.7	NULL
		NULL		NULL
NULL	NULL		NULL	495.0
10.23	3.8	4.2	Credit Card	2024-08-08 06:3
4:38	3.5-4			
""CNR8779845""	Completed	""CID7444351""	Auto Fari	
dabad Sector 15	Malviya Nagar	8.9	31.1	
NULL		NULL		NULL
NULL	NULL		NULL	2847.0
39.21	3.1	4.6	Debit Card	2024-11-03 12:2
4:59	3-3.5			
""CNR3871541""	Completed	""CID9849259""	Go Mini	
Mundka	Preet Vihar	8.9	41.0	NUL
L		NULL		NULL
NULL	NULL		NULL	282.0

```

8.36|          3.8|          4.4|          UPI|2024-11-15 05:53:
55|          3.5-4|
|""CNR2877692""|          Completed|""CID5901711""|Premier Sedan|
DLF City Court|          Nehru Place|          11.5|          20.5|
NULL|          NULL|          NULL|          NULL|
NULL|          NULL|          NULL|          656.0|
6.39|          5.0|          4.9|          Uber Wallet|2024-02-04 18:08:
04|          4.5-5|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
only showing top 20 rows

```

```

In [147... from pyspark.sql.functions import avg, count, round

stats = (filter_1.groupBy("Driver_Rating_Range").agg(count("*").ali
round(avg("Ride Distance"),2).alias("Avg Ride Distance"),ro
round(avg("Avg VTAT"),2).alias("Avg Pickup Duration"),round
stats.show()

#clear memory of intermediate dataframe
filter_1.unpersist()

```

```

+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|Driver_Rating_Range|Count|Avg Driver Rating|Avg Ride Distance|Avg B
ooking Value|Avg Pickup Duration|Avg Travel Duration|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|          3-3.5| 6697|          3.28|          26.15|
507.39|          8.55|          30.02|
|          4.5-5|23444|          4.74|          25.94|
507.81|          8.48|          30.05|
|          0-3| 745|          3.0|          25.41|
521.87|          8.66|          30.12|
|          4-4.5|46540|          4.28|          25.99|
507.57|          8.52|          30.02|
|          3.5-4|15574|          3.8|          26.08|
510.23|          8.53|          30.07|
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+

```

```

Out[147... DataFrame[Booking ID: string, Booking Status: string, Customer ID:
string, Vehicle Type: string, Pickup Location: string, Drop Locati
on: string, Avg VTAT: float, Avg CTAT: float, Cancelled Rides by C
ustomer: string, Reason for cancelling by Customer: string, Cancel
led Rides by Driver: string, Driver Cancellation Reason: string, I
ncomplete Rides: string, Incomplete Rides Reason: string, Booking
Value: float, Ride Distance: float, Driver Ratings: float, Custome
r Rating: string, Payment Method: string, Datetime: timestamp, Dri
ver_Rating_Range: string]

```

```

In [148... #BUSINESS QUERY 2 - high-velocity activity spikes: identifyig users

```

```

# three standard deviations above the standard user hourly mean. Co

#count the number of unique Customer IDs in dataset
unique_count = df.select("Customer ID").distinct().count()
total_count = df.count()
#number of enteries with repeat users
print(total_count - unique_count)

#review the Customer IDs with the most bookings
from pyspark.sql.functions import desc
repeat_bookings = (df.groupBy("Customer ID").agg(count("Booking ID"
    .orderBy(desc("Repeat_Bookings"))))
repeat_bookings.show()

#average and standard deviation number of repeat bookings
from pyspark.sql.functions import stddev
repbook_stats = repeat_bookings.agg(avg("Repeat_Bookings").alias("A
repbook_stats.show()

```

1212

Customer ID	Repeat_Bookings
CID7828101	3
CID4523979	3
CID6715450	3
CID6468528	3
CID8727691	3
CID5481002	3
CID9329900	2
CID3448023	2
CID5312396	2
CID1798550	2
CID5911647	2
CID5896172	2
CID7611095	2
CID6792844	2
CID7117070	2
CID9028135	2
CID5945148	2
CID9173522	2
CID4140572	2
CID5521293	2

only showing top 20 rows

Avg_Repeat_Bookings	StdDev_Repeat_Bookings
2.0049751243781095	0.07038807220325588

In [149... **from** pyspark.sql.functions **import** lag
from pyspark.sql.window **import** Window

```

#dataframe of users with repeat bookings that is cache

```


NULL	NULL	NULL	90.0
32.75	3.2	4.2	Uber Wallet 2024-06-03 15:5
6:24	2 2024-03-02 17:37:00	""CNR9673455""	2231.
3233333333333			
""CID1031312"" ""CNR8117971""	Completed	Auto	
	Paschim Vihar	Welcome	10.7 15.5
NULL	NULL	NULL	NULL
NULL	NULL	NULL	421.0
2.67	4.1	3.9	Credit Card 2024-12-12 21:53:
33	2	NULL	NULL
NULL			
""CID1031312"" ""CNR7339507""	Completed	Uber XL	
	Aya Nagar	Pragati Maidan	10.5 41.5
NULL	NULL	NULL	NULL
NULL	NULL	NULL	341.0
34.45	3.7	4.4	Cash 2024-12-19 15:0
1:41	2 2024-12-12 21:53:33	""CNR8117971""	161.1
35555555555556			
""CID1031683"" ""CNR8977869""	Completed	Bike	
	India Gate	Old Gurgaon	4.2 44.6
NULL	NULL	NULL	NULL
NULL	NULL	NULL	1033.0
48.06	4.3	4.7	Credit Card 2024-05-05 21:0
4:33	2	NULL	NULL
NULL			
""CID1031683"" ""CNR6300359""	No Driver Found	Go Mini	
	Netaji Subhash Place	Greater Noida	NULL NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL 2024-05-30 13:43:
59	2 2024-05-05 21:04:33	""CNR8977869""	592.65
72222222222			
""CID1032196"" ""CNR1580852""	Completed	Bike	
	Mundka	Moolchand	7.4 16.0
NULL	NULL	NULL	NULL
NULL	NULL	NULL	425.0
11.93	4.2	4.3	Cash 2024-02-04 21:2
9:14	2	NULL	NULL
NULL			
""CID1032196"" ""CNR5491824""	Incomplete	Go Mini	
	Vaishali	Sarojini Nagar	8.4 15.7
NULL	NULL	NULL	NULL
NULL	1	Vehicle Breakdown	454.0
4.47	NULL	NULL	Cash 2024-11-27 09:43:
33	2 2024-02-04 21:29:14	""CNR1580852""	7116.2
38611111111			
""CID1042090"" ""CNR5953020""	Cancelled by Driver	Auto	
	Jor Bagh	Govindpuri	6.0 NULL
NULL	NULL	NULL	1
The customer was ...	NULL	NULL	NULL
NULL	NULL	NULL	NULL 202
4-02-03 17:25:27	2	NULL	NUL
L	NULL		
""CID1042090"" ""CNR2346742""	Completed	Auto	
	Ambience Mall	Adarsh Nagar	10.0 27.5
NULL	NULL	NULL	NULL

NULL	NULL	NULL	722.0
33.59	3.7	5.0	UPI 2024-02-20 22:3
9:29	2 2024-02-03 17:25:27	""CNR5953020""	413.
2338888888889			
""CID1043706""	""CNR6910434""	Cancelled by Driver	Auto
	Sushant Lok	Bhiwadi	8.8 NULL
NULL		NULL	1
More than permitt...		NULL	NULL
NULL	NULL	NULL	NULL 202
4-03-30 18:15:29	2	NULL	NUL
L	NULL		
""CID1043706""	""CNR4389444""	Cancelled by Driver	Go Mini
	Adarsh Nagar	India Gate	8.3 NULL
NULL		NULL	1
The customer was ...		NULL	NULL
NULL	NULL	NULL	NULL 202
4-04-03 15:21:39	2 2024-03-30 18:15:29	""CNR691043	
4""	93.10277777777777		
""CID1069949""	""CNR4916911""	Completed	Bike
	Kalkaji	Uttam Nagar	12.4 19.4
NULL		NULL	NULL
NULL	NULL	NULL	363.0
30.61	5.0	3.8	UPI 2024-05-13 04:3
6:55	2	NULL	NULL
NULL			
""CID1069949""	""CNR7449757""	No Driver Found	eBike
	Kaushambi Mansarovar Park	NULL	NULL
NULL		NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL 2024-09-26 20:35:
43	2 2024-05-13 04:36:55	""CNR4916911""	
3279.98			
""CID1071374""	""CNR4067638""	Completed	Premier Sedan
	Janakpuri	AIIMS	11.1 41.1
NULL		NULL	NULL
NULL	NULL	NULL	346.0
30.72	3.9	4.2	UPI 2024-06-09 19:1
3:14	2	NULL	NULL
NULL			
""CID1071374""	""CNR2882650""	No Driver Found	Go Sedan
	Indirapuram	Bhiwadi	NULL NULL
NULL		NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL 2024-07-23 20:57:
01	2 2024-06-09 19:13:14	""CNR4067638""	1057.72
97222222223			
""CID1075408""	""CNR6504024""	No Driver Found	Auto
	Karkarduma	RK Puram	NULL NULL
NULL		NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL 2024-02-28 17:48:
48	2	NULL	NULL
NULL			
""CID1075408""	""CNR1315086""	Completed	Premier Sedan
	Faridabad Sector 15	Lok Kalyan Marg	13.7 32.6
NULL		NULL	NULL

NULL	NULL	NULL	869.0
8.93	4.3	4.3	Cash 2024-04-11 20:24:
28	2 2024-02-28	17:48:48	""CNR6504024"" 1035.59
444444444444			

only showing top 20 rows

```
In [150... #investigate the process of join in Spark
repeat_bookings_detail.explain(extended=True)
```

```
== Parsed Logical Plan ==
'Join UsingJoin(Inner, [Customer ID])
:- Project [Booking ID#57019, Booking Status#57020, Customer ID#57021, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Method#57037, Datetime#57043]
: +- Project [Date#57017, Time#57018, Booking ID#57019, Booking Status#57020, Customer ID#57021, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Method#57037, cast(concat_ws( , Date#57017, Time#57018) as timestamp) AS Datetime#57043]
:   +- Project [Date#57017, Time#57018, Booking ID#57019, Booking Status#57020, Customer ID#57021, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, cast(Avg VTAT#57025 as float) AS Avg VTAT#57038, cast(Avg CTAT#57026 as float) AS Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, cast(Booking Value#57033 as float) AS Booking Value#57040, cast(Ride Distance#57034 as float) AS Ride Distance#57041, cast(Driver Ratings#57035 as float) AS Driver Ratings#57042, Customer Rating#57036, Payment Method#57037]
:     +- Project [CASE WHEN (Date#56885 = null) THEN cast(null as string) ELSE Date#56885 END AS Date#57017, CASE WHEN (Time#56886 = null) THEN cast(null as string) ELSE Time#56886 END AS Time#57018, CASE WHEN (Booking ID#56887 = null) THEN cast(null as string) ELSE Booking ID#56887 END AS Booking ID#57019, CASE WHEN (Booking Status#56888 = null) THEN cast(null as string) ELSE Booking Status#56888 END AS Booking Status#57020, CASE WHEN (Customer ID#56889 = null) THEN cast(null as string) ELSE Customer ID#56889 END AS Customer ID#57021, CASE WHEN (Vehicle Type#56890 = null) THEN cast(null as string) ELSE Vehicle Type#56890 END AS Vehicle Type#57022, CASE WHEN (Pickup Location#56891 = null) THEN cast(null as string) ELSE Pickup Location#56891 END AS Pickup Location#57023, CASE WHEN (Drop Location#56892 = null) THEN cast(null as string) ELSE Drop Location#56892 END AS Drop Location#57024, CASE WHEN (Avg VTAT#56893 = null) THEN cast(null as float) ELSE Avg VTAT#56893 END AS Avg VTAT#57038, CASE WHEN (Avg CTAT#56894 = null) THEN cast(null as float) ELSE Avg CTAT#56894 END AS Avg CTAT#57039, CASE WHEN (Cancelled Rides by Customer#56895 = null) THEN cast(null as integer) ELSE Cancelled Rides by Customer#56895 END AS Cancelled Rides by Customer#57027, CASE WHEN (Reason for cancelling by Customer#56896 = null) THEN cast(null as string) ELSE Reason for cancelling by Customer#56896 END AS Reason for cancelling by Customer#57028, CASE WHEN (Cancelled Rides by Driver#56897 = null) THEN cast(null as integer) ELSE Cancelled Rides by Driver#56897 END AS Cancelled Rides by Driver#57029, CASE WHEN (Driver Cancellation Reason#56898 = null) THEN cast(null as string) ELSE Driver Cancellation Reason#56898 END AS Driver Cancellation Reason#57030, CASE WHEN (Incomplete Rides#56899 = null) THEN cast(null as integer) ELSE Incomplete Rides#56899 END AS Incomplete Rides#57031, CASE WHEN (Incomplete Rides Reason#56900 = null) THEN cast(null as string) ELSE Incomplete Rides Reason#56900 END AS Incomplete Rides Reason#57032, CASE WHEN (Booking Value#56901 = null) THEN cast(null as float) ELSE Booking Value#56901 END AS Booking Value#57040, CASE WHEN (Ride Distance#56902 = null) THEN cast(null as float) ELSE Ride Distance#56902 END AS Ride Distance#57041, CASE WHEN (Driver Ratings#56903 = null) THEN cast(null as float) ELSE Driver Ratings#56903 END AS Driver Ratings#57042, CASE WHEN (Customer Rating#56904 = null) THEN cast(null as float) ELSE Customer Rating#56904 END AS Customer Rating#57036, CASE WHEN (Payment Method#56905 = null) THEN cast(null as string) ELSE Payment Method#56905 END AS Payment Method#57037, CASE WHEN (Datetime#56906 = null) THEN cast(null as timestamp) ELSE Datetime#56906 END AS Datetime#57043]
```

```

tion#56891 = null) THEN cast(null as string) ELSE Pickup Location#56
891 END AS Pickup Location#57023, CASE WHEN (Drop Location#56892 = n
ull) THEN cast(null as string) ELSE Drop Location#56892 END AS Drop
Location#57024, CASE WHEN (Avg VTAT#56893 = null) THEN cast(null as
string) ELSE Avg VTAT#56893 END AS Avg VTAT#57025, CASE WHEN (Avg CT
AT#56894 = null) THEN cast(null as string) ELSE Avg CTAT#56894 END A
S Avg CTAT#57026, CASE WHEN (Cancelled Rides by Customer#56895 = nul
l) THEN cast(null as string) ELSE Cancelled Rides by Customer#56895
END AS Cancelled Rides by Customer#57027, CASE WHEN (Reason for canc
elling by Customer#56896 = null) THEN cast(null as string) ELSE Reas
on for cancelling by Customer#56896 END AS Reason for cancelling by
Customer#57028, CASE WHEN (Cancelled Rides by Driver#56897 = null) T
HEN cast(null as string) ELSE Cancelled Rides by Driver#56897 END AS
Cancelled Rides by Driver#57029, CASE WHEN (Driver Cancellation Reas
on#56898 = null) THEN cast(null as string) ELSE Driver Cancellation
Reason#56898 END AS Driver Cancellation Reason#57030, CASE WHEN (Inc
omplete Rides#56899 = null) THEN cast(null as string) ELSE Incomplet
e Rides#56899 END AS Incomplete Rides#57031, CASE WHEN (Incomplete R
ides Reason#56900 = null) THEN cast(null as string) ELSE Incomplete
Rides Reason#56900 END AS Incomplete Rides Reason#57032, CASE WHEN (
Booking Value#56901 = null) THEN cast(null as string) ELSE Booking V
alue#56901 END AS Booking Value#57033, CASE WHEN (Ride Distance#5690
2 = null) THEN cast(null as string) ELSE Ride Distance#56902 END AS
Ride Distance#57034, CASE WHEN (Driver Ratings#56903 = null) THEN ca
st(null as string) ELSE Driver Ratings#56903 END AS Driver Ratings#5
7035, CASE WHEN (Customer Rating#56904 = null) THEN cast(null as str
ing) ELSE Customer Rating#56904 END AS Customer Rating#57036, CASE W
HEN (Payment Method#56905 = null) THEN cast(null as string) ELSE Pay
ment Method#56905 END AS Payment Method#57037]
:
    +- Relation [Date#56885,Time#56886,Booking ID#56887,Book
ing Status#56888,Customer ID#56889,Vehicle Type#56890,Pickup Locatio
n#56891,Drop Location#56892,Avg VTAT#56893,Avg CTAT#56894,Cancelled
Rides by Customer#56895,Reason for cancelling by Customer#56896,Canc
elled Rides by Driver#56897,Driver Cancellation Reason#56898,Incompl
ete Rides#56899,Incomplete Rides Reason#56900,Booking Value#56901,Ri
de Distance#56902,Driver Ratings#56903,Customer Rating#56904,Payment
Method#56905] csv
+- Sort [Repeat_Bookings#59501L DESC NULLS LAST], true
    +- Filter (Repeat_Bookings#59501L > cast(1 as bigint))
        +- Aggregate [Customer ID#59704], [Customer ID#59704, count(Bo
oking ID#59702) AS Repeat_Bookings#59501L]
            +- Project [Booking ID#59702, Booking Status#59703, Custome
r ID#59704, Vehicle Type#59705, Pickup Location#59706, Drop Locatio
n#59707, Avg VTAT#59721, Avg CTAT#59722, Cancelled Rides by Custome
r#59710, Reason for cancelling by Customer#59711, Cancelled Rides by
Driver#59712, Driver Cancellation Reason#59713, Incomplete Rides#597
14, Incomplete Rides Reason#59715, Booking Value#59723, Ride Distanc
e#59724, Driver Ratings#59725, Customer Rating#59719, Payment Metho
d#59720, Datetime#59726]
                +- Project [Date#59700, Time#59701, Booking ID#59702, Bo
oking Status#59703, Customer ID#59704, Vehicle Type#59705, Pickup Lo
cation#59706, Drop Location#59707, Avg VTAT#59721, Avg CTAT#59722, C
ancelled Rides by Customer#59710, Reason for cancelling by Customer#
59711, Cancelled Rides by Driver#59712, Driver Cancellation Reason#5
9713, Incomplete Rides#59714, Incomplete Rides Reason#59715, Booking
Value#59723, Ride Distance#59724, Driver Ratings#59725, Customer Rat

```

ing#59719, Payment Method#59720, cast(concat_ws(, Date#59700, Time#59701) as timestamp) AS Datetime#59726]

+-- Project [Date#59700, Time#59701, Booking ID#59702, Booking Status#59703, Customer ID#59704, Vehicle Type#59705, Pickup Location#59706, Drop Location#59707, cast(Avg VTAT#59708 as float) AS Avg VTAT#59721, cast(Avg CTAT#59709 as float) AS Avg CTAT#59722, Cancelled Rides by Customer#59710, Reason for cancelling by Customer#59711, Cancelled Rides by Driver#59712, Driver Cancellation Reason#59713, Incomplete Rides#59714, Incomplete Rides Reason#59715, cast(Booking Value#59716 as float) AS Booking Value#59723, cast(Ride Distance#59717 as float) AS Ride Distance#59724, cast(Driver Ratings#59718 as float) AS Driver Ratings#59725, Customer Rating#59719, Payment Method#59720]

+-- Project [CASE WHEN (Date#59679 = null) THEN cast(null as string) ELSE Date#59679 END AS Date#59700, CASE WHEN (Time#59680 = null) THEN cast(null as string) ELSE Time#59680 END AS Time#59701, CASE WHEN (Booking ID#59681 = null) THEN cast(null as string) ELSE Booking ID#59681 END AS Booking ID#59702, CASE WHEN (Booking Status#59682 = null) THEN cast(null as string) ELSE Booking Status#59682 END AS Booking Status#59703, CASE WHEN (Customer ID#59683 = null) THEN cast(null as string) ELSE Customer ID#59683 END AS Customer ID#59704, CASE WHEN (Vehicle Type#59684 = null) THEN cast(null as string) ELSE Vehicle Type#59684 END AS Vehicle Type#59705, CASE WHEN (Pickup Location#59685 = null) THEN cast(null as string) ELSE Pickup Location#59685 END AS Pickup Location#59706, CASE WHEN (Drop Location#59686 = null) THEN cast(null as string) ELSE Drop Location#59686 END AS Drop Location#59707, CASE WHEN (Avg VTAT#59687 = null) THEN cast(null as string) ELSE Avg VTAT#59687 END AS Avg VTAT#59708, CASE WHEN (Avg CTAT#59688 = null) THEN cast(null as string) ELSE Avg CTAT#59688 END AS Avg CTAT#59709, CASE WHEN (Cancelled Rides by Customer#59689 = null) THEN cast(null as string) ELSE Cancelled Rides by Customer#59689 END AS Cancelled Rides by Customer#59710, CASE WHEN (Reason for cancelling by Customer#59690 = null) THEN cast(null as string) ELSE Reason for cancelling by Customer#59690 END AS Reason for cancelling by Customer#59711, CASE WHEN (Cancelled Rides by Driver#59691 = null) THEN cast(null as string) ELSE Cancelled Rides by Driver#59691 END AS Cancelled Rides by Driver#59712, CASE WHEN (Driver Cancellation Reason#59692 = null) THEN cast(null as string) ELSE Driver Cancellation Reason#59692 END AS Driver Cancellation Reason#59713, CASE WHEN (Incomplete Rides#59693 = null) THEN cast(null as string) ELSE Incomplete Rides#59693 END AS Incomplete Rides#59714, CASE WHEN (Incomplete Rides Reason#59694 = null) THEN cast(null as string) ELSE Incomplete Rides Reason#59694 END AS Incomplete Rides Reason#59715, CASE WHEN (Booking Value#59695 = null) THEN cast(null as string) ELSE Booking Value#59695 END AS Booking Value#59716, CASE WHEN (Ride Distance#59696 = null) THEN cast(null as string) ELSE Ride Distance#59696 END AS Ride Distance#59717, CASE WHEN (Driver Ratings#59697 = null) THEN cast(null as string) ELSE Driver Ratings#59697 END AS Driver Ratings#59718, CASE WHEN (Customer Rating#59698 = null) THEN cast(null as string) ELSE Customer Rating#59698 END AS Customer Rating#59719, CASE WHEN (Payment Method#59699 = null) THEN cast(null as string) ELSE Payment Method#59699 END AS Payment Method#59720]

+-- Relation [Date#59679, Time#59680, Booking ID#59681, Booking Status#59682, Customer ID#59683, Vehicle Type#59684, Pickup Location#59685, Drop Location#59686, Avg VTAT#59687, Avg CTAT#59688, Cancelled Rides by Customer#59689, Reason for cancelling by Customer#59690, Cancelled Rides by Driver#59691, Driver Cancellation Reason#59692, Incomplete Rides#59693, Incomplete Rides Reason#59694, Booking Value#59695, Ride Distance#59696, Driver Ratings#59697, Customer Rating#59698, Payment Method#59699]

9690,Cancelled Rides by Driver#59691,Driver Cancellation Reason#59692,Incomplete Rides#59693,Incomplete Rides Reason#59694,Booking Value#59695,Ride Distance#59696,Driver Ratings#59697,Customer Rating#59698,Payment Method#59699] csv

== Analyzed Logical Plan ==

Customer ID: string, Booking ID: string, Booking Status: string, Vehicle Type: string, Pickup Location: string, Drop Location: string, Avg VTAT: float, Avg CTAT: float, Cancelled Rides by Customer: string, Reason for cancelling by Customer: string, Cancelled Rides by Driver: string, Driver Cancellation Reason: string, Incomplete Rides: string, Incomplete Rides Reason: string, Booking Value: float, Ride Distance: float, Driver Ratings: float, Customer Rating: string, Payment Method: string, Datetime: timestamp, Repeat_Bookings: bigint
Project [Customer ID#57021, Booking ID#57019, Booking Status#57020, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Method#57037, Datetime#57043, Repeat_Bookings#59501L]

+-- Join Inner, (Customer ID#57021 = Customer ID#59704)

:- Project [Booking ID#57019, Booking Status#57020, Customer ID#57021, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Method#57037, Datetime#57043]

: +- Project [Date#57017, Time#57018, Booking ID#57019, Booking Status#57020, Customer ID#57021, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Method#57037, cast(concat_ws(, Date#57017, Time#57018) as timestamp) AS Datetime#57043]

: +- Project [Date#57017, Time#57018, Booking ID#57019, Booking Status#57020, Customer ID#57021, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, cast(Avg VTAT#57025 as float) AS Avg VTAT#57038, cast(Avg CTAT#57026 as float) AS Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, cast(Booking Value#57033 as float) AS Booking Value#57040, cast(Ride Distance#57034 as float) AS Ride Distance#57041, cast(Driver Ratings#57035 as float) AS Driver Ratings#57042, Customer Rating#57036, Payment Method#57037]

: +- Project [CASE WHEN (Date#56885 = null) THEN cast(null as string) ELSE Date#56885 END AS Date#57017, CASE WHEN (Time#56886 = null) THEN cast(null as string) ELSE Time#56886 END AS Time#57018, CASE WHEN (Booking ID#56887 = null) THEN cast(null as string) ELSE Booking ID#56887 END AS Booking ID#57019, CASE WHEN (Booking Status#5

```

6888 = null) THEN cast(null as string) ELSE Booking Status#56888 END
AS Booking Status#57020, CASE WHEN (Customer ID#56889 = null) THEN c
ast(null as string) ELSE Customer ID#56889 END AS Customer ID#57021,
CASE WHEN (Vehicle Type#56890 = null) THEN cast(null as string) ELSE
Vehicle Type#56890 END AS Vehicle Type#57022, CASE WHEN (Pickup Loca
tion#56891 = null) THEN cast(null as string) ELSE Pickup Location#56
891 END AS Pickup Location#57023, CASE WHEN (Drop Location#56892 = n
ull) THEN cast(null as string) ELSE Drop Location#56892 END AS Drop
Location#57024, CASE WHEN (Avg VTAT#56893 = null) THEN cast(null as
string) ELSE Avg VTAT#56893 END AS Avg VTAT#57025, CASE WHEN (Avg CT
AT#56894 = null) THEN cast(null as string) ELSE Avg CTAT#56894 END A
S Avg CTAT#57026, CASE WHEN (Cancelled Rides by Customer#56895 = nul
l) THEN cast(null as string) ELSE Cancelled Rides by Customer#56895
END AS Cancelled Rides by Customer#57027, CASE WHEN (Reason for canc
elling by Customer#56896 = null) THEN cast(null as string) ELSE Reas
on for cancelling by Customer#56896 END AS Reason for cancelling by
Customer#57028, CASE WHEN (Cancelled Rides by Driver#56897 = null) T
HEN cast(null as string) ELSE Cancelled Rides by Driver#56897 END AS
Cancelled Rides by Driver#57029, CASE WHEN (Driver Cancellation Reas
on#56898 = null) THEN cast(null as string) ELSE Driver Cancellation
Reason#56898 END AS Driver Cancellation Reason#57030, CASE WHEN (Inc
omplete Rides#56899 = null) THEN cast(null as string) ELSE Incomplet
e Rides#56899 END AS Incomplete Rides#57031, CASE WHEN (Incomplete R
ides Reason#56900 = null) THEN cast(null as string) ELSE Incomplete
Rides Reason#56900 END AS Incomplete Rides Reason#57032, CASE WHEN (
Booking Value#56901 = null) THEN cast(null as string) ELSE Booking V
alue#56901 END AS Booking Value#57033, CASE WHEN (Ride Distance#5690
2 = null) THEN cast(null as string) ELSE Ride Distance#56902 END AS
Ride Distance#57034, CASE WHEN (Driver Ratings#56903 = null) THEN ca
st(null as string) ELSE Driver Ratings#56903 END AS Driver Ratings#5
7035, CASE WHEN (Customer Rating#56904 = null) THEN cast(null as str
ing) ELSE Customer Rating#56904 END AS Customer Rating#57036, CASE W
HEN (Payment Method#56905 = null) THEN cast(null as string) ELSE Pay
ment Method#56905 END AS Payment Method#57037]
:
+- Relation [Date#56885,Time#56886,Booking ID#56887,B
ooking Status#56888,Customer ID#56889,Vehicle Type#56890,Pickup Loca
tion#56891,Drop Location#56892,Avg VTAT#56893,Avg CTAT#56894,Cancell
ed Rides by Customer#56895,Reason for cancelling by Customer#56896,C
ancelled Rides by Driver#56897,Driver Cancellation Reason#56898,Inco
mplete Rides#56899,Incomplete Rides Reason#56900,Booking Value#5690
1,Ride Distance#56902,Driver Ratings#56903,Customer Rating#56904,Pay
ment Method#56905] csv
+- Sort [Repeat_Bookings#59501L DESC NULLS LAST], true
+- Filter (Repeat_Bookings#59501L > cast(1 as bigint))
+- Aggregate [Customer ID#59704], [Customer ID#59704, coun
t(Booking ID#59702) AS Repeat_Bookings#59501L]
+- Project [Booking ID#59702, Booking Status#59703, Cust
omer ID#59704, Vehicle Type#59705, Pickup Location#59706, Drop Locat
ion#59707, Avg VTAT#59721, Avg CTAT#59722, Cancelled Rides by Custom
er#59710, Reason for cancelling by Customer#59711, Cancelled Rides b
y Driver#59712, Driver Cancellation Reason#59713, Incomplete Rides#5
9714, Incomplete Rides Reason#59715, Booking Value#59723, Ride Dista
nce#59724, Driver Ratings#59725, Customer Rating#59719, Payment Meth
od#59720, Datetime#59726]
+- Project [Date#59700, Time#59701, Booking ID#59702,
Booking Status#59703, Customer ID#59704, Vehicle Type#59705, Pickup

```

Location#59706, Drop Location#59707, Avg VTAT#59721, Avg CTAT#59722, Cancelled Rides by Customer#59710, Reason for cancelling by Customer#59711, Cancelled Rides by Driver#59712, Driver Cancellation Reason#59713, Incomplete Rides#59714, Incomplete Rides Reason#59715, Booking Value#59723, Ride Distance#59724, Driver Ratings#59725, Customer Rating#59719, Payment Method#59720, cast(concat_ws(, Date#59700, Time#59701) as timestamp) AS Datetime#59726]

+ Project [Date#59700, Time#59701, Booking ID#59702, Booking Status#59703, Customer ID#59704, Vehicle Type#59705, Pickup Location#59706, Drop Location#59707, cast(Avg VTAT#59708 as float) AS Avg VTAT#59721, cast(Avg CTAT#59709 as float) AS Avg CTAT#59722, Cancelled Rides by Customer#59710, Reason for cancelling by Customer#59711, Cancelled Rides by Driver#59712, Driver Cancellation Reason#59713, Incomplete Rides#59714, Incomplete Rides Reason#59715, cast(Booking Value#59716 as float) AS Booking Value#59723, cast(Ride Distance#59717 as float) AS Ride Distance#59724, cast(Driver Ratings#59718 as float) AS Driver Ratings#59725, Customer Rating#59719, Payment Method#59720]

+ Project [CASE WHEN (Date#59679 = null) THEN cast(null as string) ELSE Date#59679 END AS Date#59700, CASE WHEN (Time#59680 = null) THEN cast(null as string) ELSE Time#59680 END AS Time#59701, CASE WHEN (Booking ID#59681 = null) THEN cast(null as string) ELSE Booking ID#59681 END AS Booking ID#59702, CASE WHEN (Booking Status#59682 = null) THEN cast(null as string) ELSE Booking Status#59682 END AS Booking Status#59703, CASE WHEN (Customer ID#59683 = null) THEN cast(null as string) ELSE Customer ID#59683 END AS Customer ID#59704, CASE WHEN (Vehicle Type#59684 = null) THEN cast(null as string) ELSE Vehicle Type#59684 END AS Vehicle Type#59705, CASE WHEN (Pickup Location#59685 = null) THEN cast(null as string) ELSE Pickup Location#59685 END AS Pickup Location#59706, CASE WHEN (Drop Location#59686 = null) THEN cast(null as string) ELSE Drop Location#59686 END AS Drop Location#59707, CASE WHEN (Avg VTAT#59687 = null) THEN cast(null as string) ELSE Avg VTAT#59687 END AS Avg VTAT#59708, CASE WHEN (Avg CTAT#59688 = null) THEN cast(null as string) ELSE Avg CTAT#59688 END AS Avg CTAT#59709, CASE WHEN (Cancelled Rides by Customer#59689 = null) THEN cast(null as string) ELSE Cancelled Rides by Customer#59689 END AS Cancelled Rides by Customer#59710, CASE WHEN (Reason for cancelling by Customer#59690 = null) THEN cast(null as string) ELSE Reason for cancelling by Customer#59690 END AS Reason for cancelling by Customer#59711, CASE WHEN (Cancelled Rides by Driver#59691 = null) THEN cast(null as string) ELSE Cancelled Rides by Driver#59691 END AS Cancelled Rides by Driver#59712, CASE WHEN (Driver Cancellation Reason#59692 = null) THEN cast(null as string) ELSE Driver Cancellation Reason#59692 END AS Driver Cancellation Reason#59713, CASE WHEN (Incomplete Rides#59693 = null) THEN cast(null as string) ELSE Incomplete Rides#59693 END AS Incomplete Rides#59714, CASE WHEN (Incomplete Rides Reason#59694 = null) THEN cast(null as string) ELSE Incomplete Rides Reason#59694 END AS Incomplete Rides Reason#59715, CASE WHEN (Booking Value#59695 = null) THEN cast(null as string) ELSE Booking Value#59695 END AS Booking Value#59716, CASE WHEN (Ride Distance#59696 = null) THEN cast(null as string) ELSE Ride Distance#59696 END AS Ride Distance#59717, CASE WHEN (Driver Ratings#59697 = null) THEN cast(null as string) ELSE Driver Ratings#59697 END AS Driver Ratings#59718, CASE WHEN (Customer Rating#59698 = null) THEN cast(null as string) ELSE Customer Rating#59698 END AS Customer Rating#59719, CASE WHEN (Payment Method#59699 = null) THEN cast(null as str

```

ing) ELSE Payment Method#59699 END AS Payment Method#59720]
      +- Relation [Date#59679,Time#59680,Booking I
D#59681,Booking Status#59682,Customer ID#59683,Vehicle Type#59684,Pi
ckup Location#59685,Drop Location#59686,Avg VTAT#59687,Avg CTAT#5968
8,Cancelled Rides by Customer#59689,Reason for cancelling by Custome
r#59690,Cancelled Rides by Driver#59691,Driver Cancellation Reason#5
9692,Incomplete Rides#59693,Incomplete Rides Reason#59694,Booking Va
lue#59695,Ride Distance#59696,Driver Ratings#59697,Customer Rating#5
9698,Payment Method#59699] csv

```

== Optimized Logical Plan ==

```

InMemoryRelation [Customer ID#57021, Booking ID#57019, Booking Statu
s#57020, Vehicle Type#57022, Pickup Location#57023, Drop Location#57
024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#570
27, Reason for cancelling by Customer#57028, Cancelled Rides by Driv
er#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031,
Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57
041, Driver Ratings#57042, Customer Rating#57036, Payment Method#570
37, Datetime#57043, Repeat_Bookings#59501L], StorageLevel(disk, memo
ry, deserialized, 1 replicas)

```

```

  +- AdaptiveSparkPlan isFinalPlan=true

```

```

    +- == Final Plan ==

```

```

      ResultQueryStage 2

```

```

      +- *(3) Project [Customer ID#57021, Booking ID#57019, Booki
ng Status#57020, Vehicle Type#57022, Pickup Location#57023, Drop Loc
ation#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Cust
omer#57027, Reason for cancelling by Customer#57028, Cancelled Rides
by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#
57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Dist
ance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Met
hod#57037, Datetime#57043, Repeat_Bookings#59501L]

```

```

      +- *(3) BroadcastHashJoin [Customer ID#57021], [Customer
ID#59704], Inner, BuildRight, false, false

```

```

        :- *(3) Project [CASE WHEN (Booking ID#56887 = null)
THEN null ELSE Booking ID#56887 END AS Booking ID#57019, CASE WHEN (
Booking Status#56888 = null) THEN null ELSE Booking Status#56888 END
AS Booking Status#57020, CASE WHEN (Customer ID#56889 = null) THEN n
ull ELSE Customer ID#56889 END AS Customer ID#57021, CASE WHEN (Vehi
cle Type#56890 = null) THEN null ELSE Vehicle Type#56890 END AS Vehi
cle Type#57022, CASE WHEN (Pickup Location#56891 = null) THEN null E
LSE Pickup Location#56891 END AS Pickup Location#57023, CASE WHEN (D
rop Location#56892 = null) THEN null ELSE Drop Location#56892 END AS
Drop Location#57024, CASE WHEN (Avg VTAT#56893 = null) THEN null ELS
E cast(Avg VTAT#56893 as float) END AS Avg VTAT#57038, CASE WHEN (Av
g CTAT#56894 = null) THEN null ELSE cast(Avg CTAT#56894 as float) EN
D AS Avg CTAT#57039, CASE WHEN (Cancelled Rides by Customer#56895 =
null) THEN null ELSE Cancelled Rides by Customer#56895 END AS Cancel
led Rides by Customer#57027, CASE WHEN (Reason for cancelling by Cust
omer#56896 = null) THEN null ELSE Reason for cancelling by Custome
r#56896 END AS Reason for cancelling by Customer#57028, CASE WHEN (C
ancelled Rides by Driver#56897 = null) THEN null ELSE Cancelled Ride
s by Driver#56897 END AS Cancelled Rides by Driver#57029, CASE WHEN
(Driver Cancellation Reason#56898 = null) THEN null ELSE Driver Canc
ellation Reason#56898 END AS Driver Cancellation Reason#57030, CASE
WHEN (Incomplete Rides#56899 = null) THEN null ELSE Incomplete Ride
s#56899 END AS Incomplete Rides#57031, CASE WHEN (Incomplete Rides R

```

```

eason#56900 = null) THEN null ELSE Incomplete Rides Reason#56900 END
AS Incomplete Rides Reason#57032, CASE WHEN (Booking Value#56901 = n
ull) THEN null ELSE cast(Booking Value#56901 as float) END AS Bookin
g Value#57040, CASE WHEN (Ride Distance#56902 = null) THEN null ELSE
cast(Ride Distance#56902 as float) END AS Ride Distance#57041, CASE
WHEN (Driver Ratings#56903 = null) THEN null ELSE cast(Driver Rating
s#56903 as float) END AS Driver Ratings#57042, CASE WHEN (Customer R
ating#56904 = null) THEN null ELSE Customer Rating#56904 END AS Cust
omer Rating#57036, CASE WHEN (Payment Method#56905 = null) THEN null
ELSE Payment Method#56905 END AS Payment Method#57037, cast(concat_w
s( , CASE WHEN (Date#56885 = null) THEN null ELSE Date#56885 END, CA
SE WHEN (Time#56886 = null) THEN null ELSE Time#56886 END) as timest
amp) AS Datetime#57043]
      : +- *(3) Filter CASE WHEN (Customer ID#56889 = nul
l) THEN false ELSE isnotnull(Customer ID#56889) END
      : +- FileScan csv [Date#56885,Time#56886,Booking
ID#56887,Booking Status#56888,Customer ID#56889,Vehicle Type#56890,P
ickup Location#56891,Drop Location#56892,Avg VTAT#56893,Avg CTAT#568
94,Cancelled Rides by Customer#56895,Reason for cancelling by Custom
er#56896,Cancelled Rides by Driver#56897,Driver Cancellation Reason#
56898,Incomplete Rides#56899,Incomplete Rides Reason#56900,Booking V
alue#56901,Ride Distance#56902,Driver Ratings#56903,Customer Rating#
56904,Payment Method#56905] Batched: false, DataFilters: [CASE WHEN
(Customer ID#56889 = null) THEN false ELSE isnotnull(Customer ID#568
89) END], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Us
ers/ilianapeters/Documents/Documents - Iliana's MacBook Pro/Mo..., P
artitionFilters: [], PushedFilters: [], ReadSchema: struct<Date:string,Time:string,Booking ID:string,Booking Status:string,Customer ID:string,Vehicle...
      +- BroadcastQueryStage 1
      +- BroadcastExchange HashedRelationBroadcastMode(L
ist(input[0, string, true]),false), [plan_id=25824]
      +- *(2) Filter (Repeat_Bookings#59501L > 1)
      +- *(2) HashAggregate(keys=[Customer ID#5970
4], functions=[count(Booking ID#59702)], output=[Customer ID#59704,
Repeat_Bookings#59501L])
      +- AQEShuffleRead coalesced
      +- ShuffleQueryStage 0
      +- Exchange hashpartitioning(Custom
er ID#59704, 200), ENSURE_REQUIREMENTS, [plan_id=25776]
      +- *(1) HashAggregate(keys=[Cust
omer ID#59704], functions=[partial_count(Booking ID#59702)], output=
[Customer ID#59704, count#59729L])
      +- *(1) Project [CASE WHEN (B
ooking ID#59681 = null) THEN null ELSE Booking ID#59681 END AS Booki
ng ID#59702, CASE WHEN (Customer ID#59683 = null) THEN null ELSE Cust
omer ID#59683 END AS Customer ID#59704]
      +- *(1) Filter CASE WHEN (
Customer ID#59683 = null) THEN false ELSE isnotnull(Customer ID#5968
3) END
      +- FileScan csv [Bookin
g ID#59681,Customer ID#59683] Batched: false, DataFilters: [CASE WHE
N (Customer ID#59683 = null) THEN false ELSE isnotnull(Customer ID#5
9683) END], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/
Users/ilianapeters/Documents/Documents - Iliana's MacBook Pro/Mo...,
PartitionFilters: [], PushedFilters: [], ReadSchema: struct<Booking

```

```

ID:string, Customer ID:string>
    +- == Initial Plan ==
        Project [Customer ID#57021, Booking ID#57019, Booking Status#57020, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Method#57037, Datetime#57043, Repeat_Bookings#59501L]
            +- BroadcastHashJoin [Customer ID#57021], [Customer ID#59704], Inner, BuildRight, false, false
                :- Project [CASE WHEN (Booking ID#56887 = null) THEN null ELSE Booking ID#56887 END AS Booking ID#57019, CASE WHEN (Booking Status#56888 = null) THEN null ELSE Booking Status#56888 END AS Booking Status#57020, CASE WHEN (Customer ID#56889 = null) THEN null ELSE Customer ID#56889 END AS Customer ID#57021, CASE WHEN (Vehicle Type#56890 = null) THEN null ELSE Vehicle Type#56890 END AS Vehicle Type#57022, CASE WHEN (Pickup Location#56891 = null) THEN null ELSE Pickup Location#56891 END AS Pickup Location#57023, CASE WHEN (Drop Location#56892 = null) THEN null ELSE Drop Location#56892 END AS Drop Location#57024, CASE WHEN (Avg VTAT#56893 = null) THEN null ELSE cast(Avg VTAT#56893 as float) END AS Avg VTAT#57038, CASE WHEN (Avg CTAT#56894 = null) THEN null ELSE cast(Avg CTAT#56894 as float) END AS Avg CTAT#57039, CASE WHEN (Cancelled Rides by Customer#56895 = null) THEN null ELSE Cancelled Rides by Customer#56895 END AS Cancelled Rides by Customer#57027, CASE WHEN (Reason for cancelling by Customer#56896 = null) THEN null ELSE Reason for cancelling by Customer#56896 END AS Reason for cancelling by Customer#57028, CASE WHEN (Cancelled Rides by Driver#56897 = null) THEN null ELSE Cancelled Rides by Driver#56897 END AS Cancelled Rides by Driver#57029, CASE WHEN (Driver Cancellation Reason#56898 = null) THEN null ELSE Driver Cancellation Reason#56898 END AS Driver Cancellation Reason#57030, CASE WHEN (Incomplete Rides#56899 = null) THEN null ELSE Incomplete Rides#56899 END AS Incomplete Rides#57031, CASE WHEN (Incomplete Rides Reason#56900 = null) THEN null ELSE Incomplete Rides Reason#56900 END AS Incomplete Rides Reason#57032, CASE WHEN (Booking Value#56901 = null) THEN null ELSE cast(Booking Value#56901 as float) END AS Booking Value#57040, CASE WHEN (Ride Distance#56902 = null) THEN null ELSE cast(Ride Distance#56902 as float) END AS Ride Distance#57041, CASE WHEN (Driver Ratings#56903 = null) THEN null ELSE cast(Driver Ratings#56903 as float) END AS Driver Ratings#57042, CASE WHEN (Customer Rating#56904 = null) THEN null ELSE Customer Rating#56904 END AS Customer Rating#57036, CASE WHEN (Payment Method#56905 = null) THEN null ELSE Payment Method#56905 END AS Payment Method#57037, cast(concat_ws( , CASE WHEN (Date#56885 = null) THEN null ELSE Date#56885 END, CASE WHEN (Time#56886 = null) THEN null ELSE Time#56886 END) as timestamp) AS Datetime#57043]
                    : +- Filter CASE WHEN (Customer ID#56889 = null) THEN false ELSE isnotnull(Customer ID#56889) END
                        : +- FileScan csv [Date#56885, Time#56886, Booking ID#56887, Booking Status#56888, Customer ID#56889, Vehicle Type#56890, Pickup Location#56891, Drop Location#56892, Avg VTAT#56893, Avg CTAT#56894, Cancelled Rides by Customer#56895, Reason for cancelling by Customer#56896, Cancelled Rides by Driver#56897, Driver Cancellation Reason#56898, Incomplete Rides#56899, Incomplete Rides Reason#56900, Booking Value#56901, Ride Distance#56902, Driver Ratings#56903, Customer Rating#56904, Payment Method#56905]

```

```
e#56901,Ride Distance#56902,Driver Ratings#56903,Customer Rating#56904,Payment Method#56905] Batched: false, DataFilters: [CASE WHEN (Customer ID#56889 = null) THEN false ELSE isnotnull(Customer ID#56889) END], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/iliana/peters/Documents/Documents - Iliana's MacBook Pro/Mo..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<Date:string,Time:string,Booking ID:string,Booking Status:string,Customer ID:string,Vehicle...
```

```
    +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true]),false), [plan_id=25733]
```

```
    +- Filter (Repeat_Bookings#59501L > 1)
```

```
    +- HashAggregate(keys=[Customer ID#59704], functions=[count(Booking ID#59702)], output=[Customer ID#59704, Repeat_Bookings#59501L])
```

```
    +- Exchange hashpartitioning(Customer ID#59704, 200), ENSURE_REQUIREMENTS, [plan_id=25729]
```

```
    +- HashAggregate(keys=[Customer ID#59704], functions=[partial_count(Booking ID#59702)], output=[Customer ID#59704, count#59729L])
```

```
    +- Project [CASE WHEN (Booking ID#59681 = null) THEN null ELSE Booking ID#59681 END AS Booking ID#59702, CASE WHEN (Customer ID#59683 = null) THEN null ELSE Customer ID#59683 END AS Customer ID#59704]
```

```
    +- Filter CASE WHEN (Customer ID#59683 = null) THEN false ELSE isnotnull(Customer ID#59683) END
```

```
    +- FileScan csv [Booking ID#59681,Customer ID#59683] Batched: false, DataFilters: [CASE WHEN (Customer ID#59683 = null) THEN false ELSE isnotnull(Customer ID#59683) END], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/iliana/peters/Documents/Documents - Iliana's MacBook Pro/Mo..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<Booking ID:string,Customer ID:string>
```

```
== Physical Plan ==
```

```
AdaptiveSparkPlan isFinalPlan=false
```

```
+- InMemoryTableScan [Customer ID#57021, Booking ID#57019, Booking Status#57020, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Method#57037, Datetime#57043, Repeat_Bookings#59501L]
```

```
    +- InMemoryRelation [Customer ID#57021, Booking ID#57019, Booking Status#57020, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Method#57037, Datetime#57043, Repeat_Bookings#59501L], StorageLevel(disk, memory, deserialized, 1 replicas)
```

```
    +- AdaptiveSparkPlan isFinalPlan=true
```

```
    +- == Final Plan ==
```

```
        ResultQueryStage 2
```

```
        +- *(3) Project [Customer ID#57021, Booking ID#570
```

19, Booking Status#57020, Vehicle Type#57022, Pickup Location#57023, Drop Location#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Customer#57027, Reason for cancelling by Customer#57028, Cancelled Rides by Driver#57029, Driver Cancellation Reason#57030, Incomplete Rides#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Distance#57041, Driver Ratings#57042, Customer Rating#57036, Payment Method#57037, Datetime#57043, Repeat_Bookings#59501L]

```
      +- *(3) BroadcastHashJoin [Customer ID#57021],
[Customer ID#59704], Inner, BuildRight, false, false
      :- *(3) Project [CASE WHEN (Booking ID#56887
= null) THEN null ELSE Booking ID#56887 END AS Booking ID#57019, CAS
E WHEN (Booking Status#56888 = null) THEN null ELSE Booking Status#5
6888 END AS Booking Status#57020, CASE WHEN (Customer ID#56889 = nul
l) THEN null ELSE Customer ID#56889 END AS Customer ID#57021, CASE W
HEN (Vehicle Type#56890 = null) THEN null ELSE Vehicle Type#56890 EN
D AS Vehicle Type#57022, CASE WHEN (Pickup Location#56891 = null) TH
EN null ELSE Pickup Location#56891 END AS Pickup Location#57023, CAS
E WHEN (Drop Location#56892 = null) THEN null ELSE Drop Location#568
92 END AS Drop Location#57024, CASE WHEN (Avg VTAT#56893 = null) THE
N null ELSE cast(Avg VTAT#56893 as float) END AS Avg VTAT#57038, CAS
E WHEN (Avg CTAT#56894 = null) THEN null ELSE cast(Avg CTAT#56894 as
float) END AS Avg CTAT#57039, CASE WHEN (Cancelled Rides by Custome
r#56895 = null) THEN null ELSE CancelledRides by Customer#56895 END
AS Cancelled Rides by Customer#57027, CASE WHEN (Reason for cancelli
ng by Customer#56896 = null) THEN null ELSE Reason for cancelling by
Customer#56896 END AS Reason for cancelling by Customer#57028, CASE
WHEN (Cancelled Rides by Driver#56897 = null) THEN null ELSE Cancell
ed Rides by Driver#56897 END AS Cancelled Rides by Driver#57029, CAS
E WHEN (Driver Cancellation Reason#56898 = null) THEN null ELSE Driv
er Cancellation Reason#56898 END AS Driver Cancellation Reason#5703
0, CASE WHEN (Incomplete Rides#56899 = null) THEN null ELSE Incomple
te Rides#56899 END AS Incomplete Rides#57031, CASE WHEN (Incomplete
Rides Reason#56900 = null) THEN null ELSE Incomplete Rides Reason#56
900 END AS Incomplete Rides Reason#57032, CASE WHEN (Booking Value#5
6901 = null) THEN null ELSE cast(Booking Value#56901 as float) END A
S Booking Value#57040, CASE WHEN (Ride Distance#56902 = null) THEN n
ull ELSE cast(Ride Distance#56902 as float) END AS Ride Distance#570
41, CASE WHEN (Driver Ratings#56903 = null) THEN null ELSE cast(Driv
er Ratings#56903 as float) END AS Driver Ratings#57042, CASE WHEN (C
ustomer Rating#56904 = null) THEN null ELSE Customer Rating#56904 EN
D AS Customer Rating#57036, CASE WHEN (Payment Method#56905 = null)
THEN null ELSE Payment Method#56905 END AS Payment Method#57037, cas
t(concat_ws( , CASE WHEN (Date#56885 = null) THEN null ELSE Date#568
85 END, CASE WHEN (Time#56886 = null) THEN null ELSE Time#56886 END)
as timestamp) AS Datetime#57043]
```

```
      : +- *(3) Filter CASE WHEN (Customer ID#568
89 = null) THEN false ELSE isnotnull(Customer ID#56889) END
```

```
      : +- FileScan csv [Date#56885,Time#5688
6,Booking ID#56887,Booking Status#56888,Customer ID#56889,Vehicle Ty
pe#56890,Pickup Location#56891,Drop Location#56892,Avg VTAT#56893,Av
g CTAT#56894,Cancelled Rides by Customer#56895,Reason for cancelling
by Customer#56896,Cancelled Rides by Driver#56897,Driver Cancellatio
n Reason#56898,Incomplete Rides#56899,Incomplete Rides Reason#56900,
Booking Value#56901,Ride Distance#56902,Driver Ratings#56903,Custome
r Rating#56904,Payment Method#56905] Batched: false, DataFilters: [C
ASE WHEN (Customer ID#56889 = null) THEN false ELSE isnotnull(Custom
```

```

er ID#56889) END], Format: CSV, Location: InMemoryFileIndex(1 paths)
[file:/Users/ilianapeters/Documents/Documents - Iliana's MacBook Pro/Mo..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct
<Date:string,Time:string,Booking ID:string,Booking Status:string,Cus
tomer ID:string,Vehicle...
      +- BroadcastQueryStage 1
      +- BroadcastExchange HashedRelationBroadc
astMode(List(input[0, string, true]),false), [plan_id=25824]
      +- *(2) Filter (Repeat_Bookings#59501L
> 1)
      +- *(2) HashAggregate(keys=[Custome
r ID#59704], functions=[count(Booking ID#59702)], output=[Customer I
D#59704, Repeat_Bookings#59501L])
      +- AQEShuffleRead coalesced
      +- ShuffleQueryStage 0
      +- Exchange hashpartitioni
ng(Customer ID#59704, 200), ENSURE_REQUIREMENTS, [plan_id=25776]
      +- *(1) HashAggregate(k
eys=[Customer ID#59704], functions=[partial_count(Booking ID#5970
2)], output=[Customer ID#59704, count#59729L])
      +- *(1) Project [CAS
E WHEN (Booking ID#59681 = null) THEN null ELSE Booking ID#59681 END
AS Booking ID#59702, CASE WHEN (Customer ID#59683 = null) THEN null
ELSE Customer ID#59683 END AS Customer ID#59704]
      +- *(1) Filter CA
SE WHEN (Customer ID#59683 = null) THEN false ELSE isnotnull(Custome
r ID#59683) END
      +- FileScan cs
v [Booking ID#59681,Customer ID#59683] Batched: false, DataFilters:
[CASE WHEN (Customer ID#59683 = null) THEN false ELSE isnotnull(Cust
omer ID#59683) END], Format: CSV, Location: InMemoryFileIndex(1 path
s)[file:/Users/ilianapeters/Documents/Documents - Iliana's MacBook P
ro/Mo..., PartitionFilters: [], PushedFilters: [], ReadSchema: struc
t<Booking ID:string,Customer ID:string>
      +- == Initial Plan ==
      Project [Customer ID#57021, Booking ID#57019, Book
ing Status#57020, Vehicle Type#57022, Pickup Location#57023, Drop Lo
cation#57024, Avg VTAT#57038, Avg CTAT#57039, Cancelled Rides by Cus
tomer#57027, Reason for cancelling by Customer#57028, Cancelled Ride
s by Driver#57029, Driver Cancellation Reason#57030, Incomplete Ride
s#57031, Incomplete Rides Reason#57032, Booking Value#57040, Ride Di
stance#57041, Driver Ratings#57042, Customer Rating#57036, Payment M
ethod#57037, Datetime#57043, Repeat_Bookings#59501L]
      +- BroadcastHashJoin [Customer ID#57021], [Custome
r ID#59704], Inner, BuildRight, false, false
      :- Project [CASE WHEN (Booking ID#56887 = null)
THEN null ELSE Booking ID#56887 END AS Booking ID#57019, CASE WHEN (
Booking Status#56888 = null) THEN null ELSE Booking Status#56888 END
AS Booking Status#57020, CASE WHEN (Customer ID#56889 = null) THEN n
ull ELSE Customer ID#56889 END AS Customer ID#57021, CASE WHEN (Vehi
cle Type#56890 = null) THEN null ELSE Vehicle Type#56890 END AS Vehi
cle Type#57022, CASE WHEN (Pickup Location#56891 = null) THEN null E
LSE Pickup Location#56891 END AS Pickup Location#57023, CASE WHEN (D
rop Location#56892 = null) THEN null ELSE Drop Location#56892 END AS
Drop Location#57024, CASE WHEN (Avg VTAT#56893 = null) THEN null ELS
E cast(Avg VTAT#56893 as float) END AS Avg VTAT#57038, CASE WHEN (Av

```

```

g CTAT#56894 = null) THEN null ELSE cast(Avg CTAT#56894 as float) EN
D AS Avg CTAT#57039, CASE WHEN (Cancelled Rides by Customer#56895 =
null) THEN null ELSE Cancelled Rides by Customer#56895 END AS Cancel
led Rides by Customer#57027, CASE WHEN (Reason for cancelling by Cus
tomer#56896 = null) THEN null ELSE Reason for cancelling by Custome
r#56896 END AS Reason for cancelling by Customer#57028, CASE WHEN (C
ancelled Rides by Driver#56897 = null) THEN null ELSE Cancelled Ride
s by Driver#56897 END AS Cancelled Rides by Driver#57029, CASE WHEN
(Driver Cancellation Reason#56898 = null) THEN null ELSE Driver Canc
ellation Reason#56898 END AS Driver Cancellation Reason#57030, CASE
WHEN (Incomplete Rides#56899 = null) THEN null ELSE Incomplete Ride
s#56899 END AS Incomplete Rides#57031, CASE WHEN (Incomplete Rides R
eason#56900 = null) THEN null ELSE Incomplete Rides Reason#56900 END
AS Incomplete Rides Reason#57032, CASE WHEN (Booking Value#56901 = n
ull) THEN null ELSE cast(Booking Value#56901 as float) END AS Bookin
g Value#57040, CASE WHEN (Ride Distance#56902 = null) THEN null ELSE
cast(Ride Distance#56902 as float) END AS Ride Distance#57041, CASE
WHEN (Driver Ratings#56903 = null) THEN null ELSE cast(Driver Rating
s#56903 as float) END AS Driver Ratings#57042, CASE WHEN (Customer R
ating#56904 = null) THEN null ELSE Customer Rating#56904 END AS Cust
omer Rating#57036, CASE WHEN (Payment Method#56905 = null) THEN null
ELSE Payment Method#56905 END AS Payment Method#57037, cast(concat_w
s( , CASE WHEN (Date#56885 = null) THEN null ELSE Date#56885 END, CA
SE WHEN (Time#56886 = null) THEN null ELSE Time#56886 END) as timest
amp) AS Datetime#57043]
      : +- Filter CASE WHEN (Customer ID#56889 = nul
l) THEN false ELSE isnotnull(Customer ID#56889) END
      :      +- FileScan csv [Date#56885,Time#56886,Bo
oking ID#56887,Booking Status#56888,Customer ID#56889,Vehicle Type#5
6890,Pickup Location#56891,Drop Location#56892,Avg VTAT#56893,Avg CT
AT#56894,Cancelled Rides by Customer#56895,Reason for cancelling by
Customer#56896,Cancelled Rides by Driver#56897,Driver Cancellation R
eason#56898,Incomplete Rides#56899,Incomplete Rides Reason#56900,Boo
king Value#56901,Ride Distance#56902,Driver Ratings#56903,Customer R
ating#56904,Payment Method#56905] Batched: false, DataFilters: [CASE
WHEN (Customer ID#56889 = null) THEN false ELSE isnotnull(Customer I
D#56889) END], Format: CSV, Location: InMemoryFileIndex(1 paths)[fil
e:/Users/ilianapeters/Documents/Documents - Iliana's MacBook Pro/M
o..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<Da
te:string,Time:string,Booking ID:string,Booking Status:string,Custom
er ID:string,Vehicle...
      +- BroadcastExchange HashedRelationBroadcastMod
e(List(input[0, string, true]),false), [plan_id=25733]
      +- Filter (Repeat_Bookings#59501L > 1)
      +- HashAggregate(keys=[Customer ID#5970
4], functions=[count(Booking ID#59702)], output=[Customer ID#59704,
Repeat_Bookings#59501L])
      +- Exchange hashpartitioning(Customer
ID#59704, 200), ENSURE_REQUIREMENTS, [plan_id=25729]
      +- HashAggregate(keys=[Customer ID#
59704], functions=[partial_count(Booking ID#59702)], output=[Custome
r ID#59704, count#59729L])
      +- Project [CASE WHEN (Booking I
D#59681 = null) THEN null ELSE Booking ID#59681 END AS Booking ID#59
702, CASE WHEN (Customer ID#59683 = null) THEN null ELSE Customer I
D#59683 END AS Customer ID#59704]

```

```

        +- Filter CASE WHEN (Customer
ID#59683 = null) THEN false ELSE isnotnull(Customer ID#59683) END
        +- FileScan csv [Booking I
D#59681,Customer ID#59683] Batched: false, DataFilters: [CASE WHEN (
Customer ID#59683 = null) THEN false ELSE isnotnull(Customer ID#5968
3) END], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Use
rs/ilianapeters/Documents/Documents - Iliana's MacBook Pro/Mo..., Pa
rtitionFilters: [], PushedFilters: [], ReadSchema: struct<Booking I
D:string,Customer ID:string>

```

```

In [151... #Only showing bookings that are made within 24 hours
velocity_df.filter(col("Time since last Booking") < 24).show()

```

```

+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      Customer ID|      Booking ID|      Booking Status|Vehicle Type
|      Pickup Location|      Drop Location|Avg VTAT|Avg CTAT|Cancelled Ri
des by Customer|Reason for cancelling by Customer|Cancelled Rides by
Driver|Driver Cancellation Reason|Incomplete Rides|Incomplete Rides
Reason|Booking Value|Ride Distance|Driver Ratings|Customer Rating|Pa
yment Method|      Datetime|Repeat_Bookings|      Prev Booking
| Prev Booking ID|Time since last Booking|
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
|""CID1714179""|""CNR6256083""| Cancelled by Driver|      Auto
|      Noida Film City|      Noida Sector 18|      11.4|      NULL|
NULL|      NULL|      NULL|      1|
More than permitt...|      NULL|      NULL|      NULL|
NULL|      NULL|      NULL|      NULL|      NULL|202
4-04-13 17:42:55|      2|2024-04-13 07:39:46|""CNR599809
3""|      10.0525|
|""CID3211810""|""CNR7027257""|      Completed|      Bike
|      Lal Quila|      Mehrauli|      8.1|      34.3|
NULL|      NULL|      NULL|      NULL|
NULL|      NULL|      NULL|      606.0|
29.84|      4.9|      4.3|      Cash|2024-06-02 17:4
7:45|      2|2024-06-02 08:17:33|""CNR9993882""|      9.50
3333333333334|
|""CID3422015""|""CNR3149188""| Cancelled by Driver|      eBike
|      Noida Sector 62|      Pragati Maidan|      10.8|      NULL|
NULL|      NULL|      NULL|      1|
Customer related ...|      NULL|      NULL|      NULL|
NULL|      NULL|      NULL|      NULL|      NULL|202
4-11-26 11:55:37|      2|2024-11-25 15:20:11|""CNR375903
1""|      20.590555555555557|

```

["CID4021971"]	["CNR6835017"]	Completed	Auto
Kashmere Gate ISBT	Keshav Puram	11.2 24.1	
NULL	NULL	NULL	NULL
NULL	NULL	NULL	488.0
36.95	3.8	4.7	UPI 2024-05-29 17:0
3:56	2 2024-05-29 15:20:36	["CNR2309115"]	1.722
222222222223			
["CID4061291"]	["CNR9511398"]	Incomplete	Bike
Preet Vihar	Gurgaon Sector 56	2.4 29.1	
NULL	NULL	NULL	NULL
NULL	1	Vehicle Breakdown	934.0
18.59	NULL	NULL	Cash 2024-02-27 16:4
7:28	2 2024-02-27 05:45:03	["CNR5208205"]	11.04
0277777777778			
["CID7483463"]	["CNR8503014"]	Cancelled by Driver	Go Sedan
Golf Course Road	Ardee City	9.1 NULL	
NULL	NULL	NULL	1
The customer was ...	NULL	NULL	NULL
NULL	NULL	NULL	NULL 202
4-06-03 09:39:10	2 2024-06-02 21:53:40	["CNR173305	
1	11.758333333333333		
["CID7939117"]	["CNR9255555"]	Cancelled by Cust...	Auto
Greater Noida	Lajpat Nagar	10.3 NULL	
1	Driver is not mov...	NULL	NULL
NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL 2024-12-24 19:45:
48	2 2024-12-24 12:32:49	["CNR9329029"]	7.2163
88888888889			
["CID8050068"]	["CNR3772127"]	Completed	Go Mini
Nehru Place	Adarsh Nagar	12.6 22.5	
NULL	NULL	NULL	NULL
NULL	NULL	NULL	733.0
8.17	4.2	5.0	Credit Card 2024-10-28 18:00:
54	2 2024-10-27 19:02:55	["CNR6096754"]	22.966
38888888889			

```
In [152... velocity_df.filter(col("Time since last Booking") < 24).select("Cus
"Tim
```

Customer ID	Reason for cancelling by Customer	Driver Cancellation Reason	Time since last Booking	Drive
""CID1714179""	NULL			More
than permitted people in there		10.0525		
""CID3211810""	NULL			NULL
9.503333333333334				
""CID3422015""	NULL			Custo
mer related issue		20.590555555555557		
""CID4021971""	NULL			NULL
1.722222222222223				
""CID4061291""	NULL			NULL
11.040277777777778				
""CID7483463""	NULL			The c
ustomer was coughing/sick		11.758333333333333		
""CID7939117""	Driver is not moving towards pickup location			NULL
7.216388888888889				
""CID8050068""	NULL			NULL
22.966388888888889				

```
In [153... #clear memory of intermediate dataframe
repeat_bookings_detail.unpersist()
```

```
Out[153... DataFrame[Customer ID: string, Booking ID: string, Booking Status:
string, Vehicle Type: string, Pickup Location: string, Drop Locati
on: string, Avg VTAT: float, Avg CTAT: float, Cancelled Rides by C
ustomer: string, Reason for cancelling by Customer: string, Cancel
led Rides by Driver: string, Driver Cancellation Reason: string, I
ncomplete Rides: string, Incomplete Rides Reason: string, Booking
Value: float, Ride Distance: float, Driver Ratings: float, Custome
r Rating: string, Payment Method: string, Datetime: timestamp, Rep
eat_Bookings: bigint]
```

2b. Dataframe Query Finalisation

Below is the final code for the two business queries in Spark Dataframe API. It is condensed to remove any of the workings and investigative processes done in 2a. This code will be used for the comparative analysis in Part B.

```
In [154... %%time

from pyspark.sql.functions import lag, col, when, count, avg, count
from pyspark.sql.window import Window

#Query 1
query_1 = df.dropna(subset=["Driver Ratings"]).cache()
query_1.count()
query_1 = query_1.withColumn("Driver_Rating_Range",when(col("Driver
when(col("Driver Ratings")<=3.5, "3-
```

```

                                when(col("Driver Ratings")<=4.5, "4-
stats = (query_1.groupBy("Driver_Rating_Range").agg(count("*").alia
            round(avg("Ride Distance"),2).alias("Avg Ride Distance"),ro
            round(avg("Avg VTAT"),2).alias("Avg Pickup Duration"),round
print("The Uber distance, timing and price statistics seperated per
stats.show()
query_1.unpersist()

#Query 2
query_2_repeat = (df.groupBy("Customer ID").agg(count("Booking ID")
            .orderBy(desc("Repeat_Bookings"))))
from pyspark.sql.functions import stddev
repbook_stats = query_2_repeat.agg(avg("Repeat_Bookings").alias("Av
            alias("StdDev_Repeat_Bookings"))
print("The average and standard devaition of repeat bookings made i
repbook_stats.show()
repeat_bookings_detail = df.join(query_2_repeat, on="Customer ID",
repeat_bookings_detail.count()
datetime_window = Window.partitionBy("Customer ID").orderBy("Dateti
velocity_df = (repeat_bookings_detail.withColumn("Prev Booking", la
            .withColumn("Time since last Booking", (col("Datetime").cast("lo
print("The Customer ID with multiple bookings made within 24 hours,
velocity_df.filter(col("Time since last Booking") < 24).select("Cus
            "Driver Cancellation Reason", "Time sinc
repeat_bookings_detail.unpersist()

```

The Uber distance, timing and price statistics seperated per rating given by Customer

	Driver_Rating_Range	Count	Avg Driver Rating	Avg Ride Distance	Avg B ooking Value	Avg Pickup Duration	Avg Travel Duration
	3-3.5	6697	3.28	26.15			
507.39		8.55	30.02				
	4.5-5	23444	4.74	25.94			
507.81		8.48	30.05				
	0-3	745	3.0	25.41			
521.87		8.66	30.12				
	4-4.5	46540	4.28	25.99			
507.57		8.52	30.02				
	3.5-4	15574	3.8	26.08			
510.23		8.53	30.07				

The average and standard devaition of repeat bookings made in 2024

Avg_Repeat_Bookings	StdDev_Repeat_Bookings
2.0049751243781095	0.07038807220325588

The Customer ID with multiple bookings made within 24 hours, with bo
oking status, reasoning and time between bookings

```

+-----+-----+-----+
+-----+-----+-----+
-----+
|Customer ID      |Booking Status      |Reason for cancelling by Cus
tomer            |Driver Cancellation Reason      |Time since last
Booking|
+-----+-----+-----+
+-----+-----+-----+
-----+
|""CID1714179""|Cancelled by Driver |NULL
|More than permitted people in there|10.0525      |
|""CID3211810""|Completed          |NULL
|NULL          |9.5033333333333334  |
|""CID3422015""|Cancelled by Driver |NULL
|Customer related issue          |20.590555555555557  |
|""CID4021971""|Completed          |NULL
|NULL          |1.7222222222222223  |
|""CID4061291""|Incomplete         |NULL
|NULL          |11.040277777777778  |
|""CID7483463""|Cancelled by Driver |NULL
|The customer was coughing/sick   |11.758333333333333  |
|""CID7939117""|Cancelled by Customer|Driver is not moving towards
pickup location|NULL          |7.216388888888888
9      |
|""CID8050068""|Completed          |NULL
|NULL          |22.966388888888889  |
+-----+-----+-----+
+-----+-----+-----+
-----+

```

CPU times: user 13.5 ms, sys: 9.93 ms, total: 23.4 ms
Wall time: 787 ms

```

Out[154...] DataFrame[Customer ID: string, Booking ID: string, Booking Status:
string, Vehicle Type: string, Pickup Location: string, Drop Locati
on: string, Avg VTAT: float, Avg CTAT: float, Cancelled Rides by C
ustomer: string, Reason for cancelling by Customer: string, Cancel
led Rides by Driver: string, Driver Cancellation Reason: string, I
ncomplete Rides: string, Incomplete Rides Reason: string, Booking
Value: float, Ride Distance: float, Driver Ratings: float, Custome
r Rating: string, Payment Method: string, Datetime: timestamp, Rep
eat_Bookings: bigint]

```

3. Spark SQL Implementation

Implement the same query, or a functionally equivalent version, using Spark SQL

```

In [155...] %%time
df.createOrReplaceTempView("df_sql")

#Using similar code with SQL to complete query 1
query_1_sql = spark.sql("""SELECT CASE
    WHEN `Driver Ratings` <= 3.0 THEN '0-3'
    WHEN `Driver Ratings` <= 3.5 THEN '3-3.5'
    WHEN `Driver Ratings` <= 4.0 THEN '3.5-4'

```

```

        WHEN `Driver Ratings` <= 4.5 THEN '4-4.5'
        WHEN `Driver Ratings` <= 5.0 THEN '4.5-5'
        END AS Driver_Rating_Range,
        COUNT(*) AS Count, ROUND(AVG(`Driver Ratings`), 2) AS `Avg Driv
        ROUND(AVG(`Ride Distance`), 2) AS `Avg Ride Distance`, ROUND(AV
        ROUND(AVG(`Avg VTAT`), 2) AS `Avg Pickup Duration`, ROUND(AVG(`A
        FROM df_sql WHERE `Driver Ratings` IS NOT NULL GROUP BY Driver_

print("The Uber distance, timing and price statistics separated per
query_1_sql.show()

#Using similar code with SQL to complete query 2

#create temp view for the subset of data that has repeat bookings
spark.sql("""SELECT `Customer ID`, COUNT(`Booking ID`) AS Repeat_Bo
        GROUP BY `Customer ID` HAVING COUNT(`Booking ID`) > 1 ORDER BY

#get stats of average and standard deviation for customers with rep
repbook_stats_sql = spark.sql(""" SELECT AVG(Repeat_Bookings) AS Av
        FROM query_2_repeat """)
print("The average and standard deviation of repeat bookings made i
repbook_stats_sql.show()

#using an CTE to create dataset based on repeat customers and the t
velocity_sql = spark.sql("""WITH joined AS (SELECT s.*, r.Repeat_Bo
        INNER JOIN query_2_repeat r ON s.`Customer ID` = r.`Customer ID
        windowed AS (SELECT *, LAG(Datetime) OVER (PARTITION BY `Custom
        `Prev Booking`, LAG(`Booking ID`) OVER (PARTITION BY `Customer
        `Prev Booking ID` FROM joined) SELECT *, (CAST(Datetime AS LONG
        / 3600 AS `Time since last Booking` FROM windowed""")

velocity_sql.createOrReplaceTempView("velocity")

print("The Customer ID with multiple bookings made within 24 hours,
spark.sql("""SELECT `Customer ID`, `Booking Status`, `Reason for ca
        `Time since last Booking` FROM velocity WHERE `Time since last

```

The Uber distance, timing and price statistics separated per rating given by Customer

	Driver_Rating_Range	Count	Avg Driver Rating	Avg Ride Distance	Avg B ooking Value	Avg Pickup Duration	Avg Travel Duration
507.39	3-3.5	6697	3.28	26.15			
		8.55	30.02				
507.81	4.5-5	23444	4.74	25.94			
		8.48	30.05				
521.87	0-3	745	3.0	25.41			
		8.66	30.12				
507.57	4-4.5	46540	4.28	25.99			
		8.52	30.02				
510.23	3.5-4	15574	3.8	26.08			
		8.53	30.07				

```

-----+-----+-----+
The average and standard deviation of repeat bookings made in 2024
+-----+-----+
|Avg_Repeat_Bookings|StdDev_Repeat_Bookings|
+-----+-----+
| 2.0049751243781095| 0.07038807220325588|
+-----+-----+

The Customer ID with multiple bookings made within 24 hours, with bo
oking status, reasoning and time between bookings
+-----+-----+
-----+
|Customer ID      |Booking Status      |Reason for cancelling by Cus
tomer            |Driver Cancellation Reason      |Time since last
Booking|
+-----+-----+
-----+
|""CID1714179""|Cancelled by Driver |NULL
|More than permitted people in there|10.0525      |
|""CID3211810""|Completed          |NULL
|NULL          |9.5033333333333334  |
|""CID3422015""|Cancelled by Driver |NULL
|Customer related issue          |20.590555555555557  |
|""CID4021971""|Completed          |NULL
|NULL          |1.7222222222222223  |
|""CID4061291""|Incomplete         |NULL
|NULL          |11.040277777777778  |
|""CID7483463""|Cancelled by Driver |NULL
|The customer was coughing/sick   |11.758333333333333  |
|""CID7939117""|Cancelled by Customer|Driver is not moving towards
pickup location|NULL          |7.216388888888888
9          |
|""CID8050068""|Completed          |NULL
|NULL          |22.966388888888889  |
+-----+-----+
-----+

```

CPU times: user 3.15 ms, sys: 1.38 ms, total: 4.53 ms
Wall time: 861 ms

4. Result Validation

Demonstrate that both implementations produce equivalent analytical results. Where minor differences occur due to sorting, formatting, or floating-point precision, provide a brief explanation.

The two business queries were chosen to provide information relating to customer satisfaction and if there were any common themes relating to ratings given by customers and repeat bookings. They were sufficiently complicated due to the additional data operations to group per rating range

and timing between bookings per customer. Additionally, mathematical operations were required to calculate averages and time-based data for the queries. Various operations and functions were used with both Dataframe API and SQL, however Dataframe API was used to form the structure of the queries and was translated into SQL.

The output of the queries is impacted by the ordering of the code, as this impacts how much data is utilised during the operation. For example, with operations that only require a single data column, like `withColumn()`, and therefore only use that data during the operation, which makes it very efficient. Additionally, Spark projects necessary columns back through the code and therefore performs the operation on only those columns. In the second query, the final output is only with 5 columns with the information on multiple bookings. This would be projected back through the previous operations, like join and window function, to optimise the operating power as much as possible. Finally, the join operation was performed using `BroadcastHashJoin`, determined through the `explain()` function. Since the join was performed on data that was already filtered, only using two columns for this join, this type was used as it is ideal for smaller datasets that can fit in the memory; larger datasets a Sort-Merge join may have been used.

Even though Dataframe API was used initially, the SQL code is inherently easier to read due to the descriptive nature of the queries. Additionally, the queries can be combined into a single code rather than multiple lines with the Dataframe API. This also increases the computing power of SQL; the time to complete the on my CPU Dataframe API query was 1.9s compared to 1.7s with SQL query. Comparing the output of the two versions, the finalised data is identical including floating point precision when calculating the average and standard deviation of repeat booking numbers. Additionally, the sorting between the two output tables is also identical.

Part B: System perspective and performance analysis

Part B shifts the focus from "does the code work" to "why does it work this way." You will need to demonstrate an understanding of Spark's internal execution model by gathering empirical data and producing a written analysis

1. Partitioning Strategy

Investigate the partitioning strategy with hash and range partitioning using the high cardinality column - Booking ID.

1a. Hash Partitioning

```
In [156... from pyspark.sql.functions import spark_partition_id
```

```

from pyspark.sql.functions import min, max, avg, stddev

for partitions in range(1, 7):
    df_hashed = df.repartition(partitions, "Booking ID")
    hash_partition_counts = (df_hashed.withColumn("Booking ID", spa
        agg(count("*").alias("row_count"))))
    hash_skew_metrics = hash_partition_counts.select(min("row_count")
        avg("row_count").alias("avg_partition_rows"),stddev("row_co
    imbalance_ratio = (hash_partition_counts.select((max("row_count"
        .collect()[0]["imbalance_ratio"]))

    print(f"Partitions: {partitions} | "f"Hash Imbalance ratio: {im

```

```

Partitions: 1 | Hash Imbalance ratio: 1.000
Partitions: 2 | Hash Imbalance ratio: 1.007
Partitions: 3 | Hash Imbalance ratio: 1.005
Partitions: 4 | Hash Imbalance ratio: 1.015
Partitions: 5 | Hash Imbalance ratio: 1.010
Partitions: 6 | Hash Imbalance ratio: 1.015

```

In [157... *#three partitions used as this is still a suitable number to sepeara*
the dat enteries per partition senesible and meaningful. The data
increase overhead while still able to process the data
 partitions = 3

```

df_hashed = df.repartition(partitions, "Booking ID")
hash_partition_counts = (df_hashed.withColumn("Booking ID", spark_p
    agg(count("*").alias("row_count"))))
hash_skew_metrics = hash_partition_counts.select(min("row_count").a
    avg("row_count").alias("avg_partition_rows"),stddev("row_co

hash_partition_counts.show()
hash_skew_metrics.show()

```

Booking ID	row_count
0	49854
1	50122
2	50024

min_partition_rows	max_partition_rows	avg_partition_rows	stddev_partition_rows
49854	50122	50000.0	135.602

1b. Range Partitioning

In [158... *#same number of partitions as ideal hash partition*

```
df_range = df.repartitionByRange(partitions, "Booking ID")

df_range.withColumn("Booking ID", spark_partition_id()).groupBy("Bo
```

Booking ID	Row Count
0	49272
1	49861
2	50867

```
In [159... range_partition_counts = (df_range.withColumn("Booking ID", spark_p
                                agg(count("*").alias("row_count")))
range_skew_metrics = range_partition_counts.select(min("row_count")
                                                    avg("row_count").alias("avg_partition_rows"),stddev("row_co
imbalance_ratio = range_partition_counts.select((max("row_count") /

range_partition_counts.show()
range_skew_metrics.show()
imbalance_ratio.show()
```

Booking ID	row_count
0	47013
1	55649
2	47338

min_partition_rows	max_partition_rows	avg_partition_rows	stddev_partition_rows
34098240588	47078	53354	50000.0
			3160.22

imbalance_ratio
1.2504641499425337

```
In [160... #Investigate if there is different ideal number of partitions for R
#each iteration, compared to Hash which is reproducible
for partitions in range(1, 7):
    df_range = df.repartitionByRange(partitions, "Booking ID")
    range_partition_counts = (df_range.withColumn("Booking ID", spa
                                                    agg(count("*").alias("row_count")))
    range_skew_metrics = range_partition_counts.select(min("row_cou
```

```

    avg("row_count").alias("avg_partition_rows"), stddev("row_co
    imbalance_ratio = (range_partition_counts.select((max("row_coun
    .collect()[0]["imbalance_ratio"]))

    print(f"Partitions: {partitions} | "f"Range Imbalance ratio: {i

```

```

Partitions: 1 | Range Imbalance ratio: 1.000
Partitions: 2 | Range Imbalance ratio: 1.103
Partitions: 3 | Range Imbalance ratio: 1.126
Partitions: 4 | Range Imbalance ratio: 1.148
Partitions: 5 | Range Imbalance ratio: 1.301
Partitions: 6 | Range Imbalance ratio: 1.214

```

```

In [161... for partitions in range(1, 7):
    df_range = df.repartitionByRange(partitions, "Booking ID")
    range_partition_counts = (df_range.withColumn("Booking ID", spa
    agg(count("*").alias("row_count")))
    range_skew_metrics = range_partition_counts.select(min("row_cou
    avg("row_count").alias("avg_partition_rows"), stddev("row_co
    imbalance_ratio = (range_partition_counts.select((max("row_coun
    .collect()[0]["imbalance_ratio"]))

    print(f"Partitions: {partitions} | "f"Range Imbalance ratio: {i

```

```

Partitions: 1 | Range Imbalance ratio: 1.000
Partitions: 2 | Range Imbalance ratio: 1.228
Partitions: 3 | Range Imbalance ratio: 1.265
Partitions: 4 | Range Imbalance ratio: 1.143
Partitions: 5 | Range Imbalance ratio: 1.058
Partitions: 6 | Range Imbalance ratio: 1.174

```

1c Comparative Analysis

The imbalance ratio, the ratio of maximum number of rows to minimum, was used to investigate skew and ensure the partitioning was as balanced as possible. For the hash partitioning, it was 3 and this is sensible for the dataset size being not overly large but still would benefit from parallel computing. Hash partitioning did have some skew but with a standard deviation between the partitions of 135.6 and an imbalance ratio of 1.005 (which is close to 1.0), the data was distributed evenly. Since Booking ID is set as random and not in an order, hash partitioning works relatively well with this dataset.

Range partitioning with the same number of partitions shows higher skew effects, with a standard deviation of 1528.1 and imbalance ratio of 1.012. This could lead to an impact with query performance, as a partition with large number of rows would require longer processing time compared to a smaller partition. Since the queries would require all rows to be reviewed and using Booking ID to partition, having higher skew could lead to poorer performance. Interestingly, since the range partitioning estimates the boundaries between partitions each integration, these skew values change each run. This is compared to hash partitioning, which is based on a deterministic mathematical function and reproducible. Above you can see the example of

the for loop to investigate imbalance ratios for range partitioning with the ideal partition changing each time. Since the Booking ID is random and not based on a sequential order, this makes range partitioning slightly more uncertain with more variation. Therefore, with this dataset and using Booking ID for partitioning, hash partitioning may be better suited.

2. Execution time benchmarking

Using %%time, the DataFrame API and SQL was run five times and manually the total CPU times recorded below. Information about the operating system was also recorded using the below code.

API	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Run 4 (ms)	Run 5 (ms)
DataFrame	27.6	19.7	16.5	19.6	16.5
SparkSQL	8.42	5.9	7.93	7.03	6.0

```
In [162... import platform
import multiprocessing
import pyspark
import psutil

# Python / PySpark
print(f"Python version:      {platform.python_version()}")
print(f"PySpark version:      {pyspark.__version__}")
print(f"Spark master:            {spark.sparkContext.master}")

# Operating system / machine
print(f"Operating system:      {platform.system()} {platform.release}")
print(f"Architecture:           {platform.machine()}")
print(f"CPU cores available:      {multiprocessing.cpu_count()}")
print(f"System memory: {psutil.virtual_memory().total / (1024**3):.2f} GB")

Python version:      3.13.5
PySpark version:      4.2.0
Spark master:         local[*]
Operating system:      Darwin 25.5.0
Architecture:         arm64
CPU cores available:   10
System memory: 16.0 GB
```

Based on the timing analysis, the DataFrame API has more than twice the amount of CPU time to process compared to SQL for the same query. This was mainly due to the code itself, as both API types used the same Spark optimiser, which translates the code into logical plans that are then processed by the Catalyst optimiser. One main reason for the increase in time would be the use of caching with the Dataframe API. When a dataset is cached and materialised, it requires an additional Spark operation which is not present in the SQL code. Additionally, the GroupBy with query_2_repeat may be seen as an intermediate step that is not present in the SQL code, further separating

the execution time between the APIs. Python/RDD lineage and serialisation overhead would not impact timing for the above codes, which do not require row by row applications with python but instead use Spark functions to optimise the execution.

3. Execution plan interpretation

Select one join or aggregation operation from Part A implementation. The DataFrame API is recommended.

Using the `explain()` operation, the non-truncated output from the `repeat_bookings_detail` join is shown below; only the physical plan is reviewed in this discussion. Annotations are added to identify the Shuffle Exchange and operator preceding in both the initial and final plan. In this join, the original dataset is joined based on Customer ID with a dataset formed by `GroupBy` with only the Customer IDs with multiple books. Since this `explain()` focuses on both join and aggregation operators, the shuffle explanation will be on the initial aggregation only.

In the plan, there is a `HashAggregate` preceding and following the `Shuffle Exchange`. The first `HashAggregate` computes a partial count, meaning while the second computes a final count with the shuffle in the middle. This means that the first counts are per partition, which is defaulted to 200 as per `spark.sql.shuffle.partitions` setting, then the shuffle occurs that moves all rows that share a hash key (Customer ID) together, then the final count occurs with the final dataset. The partial count preceding the shuffle allows for optimisation, as only the important factors within this join are the Customer ID and number of times that occurs, therefore this is the only data used and not the entire row. This reduction in data required in the operation leads to a reduction in the network I/O. An impact on task scheduling is the shuffle is the midway point, with operations occurring afterwards waiting on the shuffle to occur. Additionally, the default partition is 200 and from the previous work in B2, this dataset be separated into a lot less partitions and still be suitable. Therefore, the default shuffle could lead to increased overhead for smaller datasets. However, in the final plan, there is `'+- AQEShuffleRead coalesced'`, which highlights the small amount of data per partition and combines them, reducing the overall number of partitions automatically and reducing overhead for small datasets.

To eliminate the need for the shuffle by pre-partitioning using bucketing with specific number of partitions or setting the number of partitions in the `spark.sql.shuffle.partitions` manually. Bucketing requires a separate table of data and doesn't change the Physical Plan below. Similarly, setting the number of partitions setting doesn't remove shuffle but does make it more efficient.

```

In [ ]: == Parsed Logical Plan ==
'Join UsingJoin(Inner, [Customer ID])
:- Project [Booking ID#11147, Booking Status#11148, Customer ID#111
: +- Project [Date#11145, Time#11146, Booking ID#11147, Booking St
: +- Project [Date#11145, Time#11146, Booking ID#11147, Booking
: +- Project [CASE WHEN (Date#11013 = null) THEN cast(null a
: +- Relation [Date#11013,Time#11014,Booking ID#11015,Boo
+- Sort [Repeat_Bookings#13629L DESC NULLS LAST], true
+- Filter (Repeat_Bookings#13629L > cast(1 as bigint))
+- Aggregate [Customer ID#13832], [Customer ID#13832, count(B
+- Project [Booking ID#13830, Booking Status#13831, Custom
+- Project [Date#13828, Time#13829, Booking ID#13830, B
+- Project [Date#13828, Time#13829, Booking ID#13830
+- Project [CASE WHEN (Date#13807 = null) THEN ca
+- Relation [Date#13807,Time#13808,Booking ID#

== Analyzed Logical Plan ==
Customer ID: string, Booking ID: string, Booking Status: string, Ve
Project [Customer ID#11149, Booking ID#11147, Booking Status#11148,
+- Join Inner, (Customer ID#11149 = Customer ID#13832)
:- Project [Booking ID#11147, Booking Status#11148, Customer ID#
: +- Project [Date#11145, Time#11146, Booking ID#11147, Booking
: +- Project [Date#11145, Time#11146, Booking ID#11147, Book
: +- Project [CASE WHEN (Date#11013 = null) THEN cast(nul
: +- Relation [Date#11013,Time#11014,Booking ID#11015,
+- Sort [Repeat_Bookings#13629L DESC NULLS LAST], true
+- Filter (Repeat_Bookings#13629L > cast(1 as bigint))
+- Aggregate [Customer ID#13832], [Customer ID#13832, coun
+- Project [Booking ID#13830, Booking Status#13831, Cus
+- Project [Date#13828, Time#13829, Booking ID#13830
+- Project [Date#13828, Time#13829, Booking ID#13
+- Project [CASE WHEN (Date#13807 = null) THEN
+- Relation [Date#13807,Time#13808,Booking

== Optimized Logical Plan ==
InMemoryRelation [Customer ID#11149, Booking ID#11147, Booking Stat
+- AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
ResultQueryStage 2
+- *(3) Project [Customer ID#11149, Booking ID#11147, Book
+- *(3) BroadcastHashJoin [Customer ID#11149], [Custome
:- *(3) Project [CASE WHEN (Booking ID#11015 = null)
: +- *(3) Filter CASE WHEN (Customer ID#11017 = nul
: +- FileScan csv [Date#11013,Time#11014,Booking
+- BroadcastQueryStage 1
+- BroadcastExchange HashedRelationBroadcastMode(
+- *(2) Filter (Repeat_Bookings#13629L > 1)
+- *(2) HashAggregate(keys=[Customer ID#138
+- AQEShuffleRead coalesced
+- ShuffleQueryStage 0
+- Exchange hashpartitioning(Custo
+- *(1) HashAggregate(keys=[Cus
+- *(1) Project [CASE WHEN (
+- *(1) Filter CASE WHEN
+- FileScan csv [Booki

+- == Initial Plan ==

```

```

Project [Customer ID#11149, Booking ID#11147, Booking Stat
+- BroadcastHashJoin [Customer ID#11149], [Customer ID#138
:- Project [CASE WHEN (Booking ID#11015 = null) THEN nu
: +- Filter CASE WHEN (Customer ID#11017 = null) THEN
: +- FileScan csv [Date#11013,Time#11014,Booking ID#
+- BroadcastExchange HashedRelationBroadcastMode(List(i
+- Filter (Repeat_Bookings#13629L > 1)
+- HashAggregate(keys=[Customer ID#13832], functi
+- Exchange hashpartitioning(Customer ID#13832
+- HashAggregate(keys=[Customer ID#13832],
+- Project [CASE WHEN (Booking ID#13809
+- Filter CASE WHEN (Customer ID#1381
+- FileScan csv [Booking ID#13809,

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- InMemoryTableScan [Customer ID#11149, Booking ID#11147, Booking
+- InMemoryRelation [Customer ID#11149, Booking ID#11147, Boo
+- AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
ResultQueryStage 2
+- *(3) Project [Customer ID#11149, Booking ID#11
+- *(3) BroadcastHashJoin [Customer ID#11149],
:- *(3) Project [CASE WHEN (Booking ID#1101
: +- *(3) Filter CASE WHEN (Customer ID#11
: +- FileScan csv [Date#11013,Time#1101
+- BroadcastQueryStage 1
+- BroadcastExchange HashedRelationBroad
+- *(2) Filter (Repeat_Bookings#13629
#Part B, 3. requirement. In the Physical Plan, this is the opera
+- *(2) HashAggregate(keys=[Custom
+- AQEShuffleRead coalesced
+- ShuffleQueryStage 0
#Part B, 3. requirement. In the Physical Plan, below is the Exc
+- Exchange hashpartition
#Part B, 3. requirement. In the Physical Plan, this is the opera
+- *(1) HashAggregate(
+- *(1) Project [CA
+- *(1) Filter C
+- FileScan c

+- == Initial Plan ==
Project [Customer ID#11149, Booking ID#11147, Boo
+- BroadcastHashJoin [Customer ID#11149], [Custom
:- Project [CASE WHEN (Booking ID#11015 = null
: +- Filter CASE WHEN (Customer ID#11017 = nu
: +- FileScan csv [Date#11013,Time#11014,B
+- BroadcastExchange HashedRelationBroadcastMo
+- Filter (Repeat_Bookings#13629L > 1)
#Part B, 3. requirement. In the initial plan, the HashAgrregate
+- HashAggregate(keys=[Customer ID#13832
#Part B, 3. requirement. The shuffle exchange is below
+- Exchange hashpartitioning(Customer
#Part B, 3. requirement. In the initial plan, the HashAgrregate
+- HashAggregate(keys=[Customer ID
+- Project [CASE WHEN (Booking
+- Filter CASE WHEN (Custome

```

4. Spark Web UI

You need to analyse the DAG (Directed Acyclic Graph) visualation generated by your DataFrame or SparkSQL implementation using Spark Web UI. A DAG visually represents the sequence of Spark operations, their dependencies, and the stage boundaries created during execution.

```
In [164... print(spark.sparkContext.uiWebViewUrl)
```

http://192.168.1.118:4040

The following is the DAG from the code discussed in Part B Section 3:
`repeat_bookings_detail = df.join(repeat_bookings, on="Customer ID", how="inner").cache` and `repeat_bookings_detail.count()`

Details for Job 482

Status: SUCCEEDED

Submitted: 2026/09/20 12:26:17

Duration: 10 ms

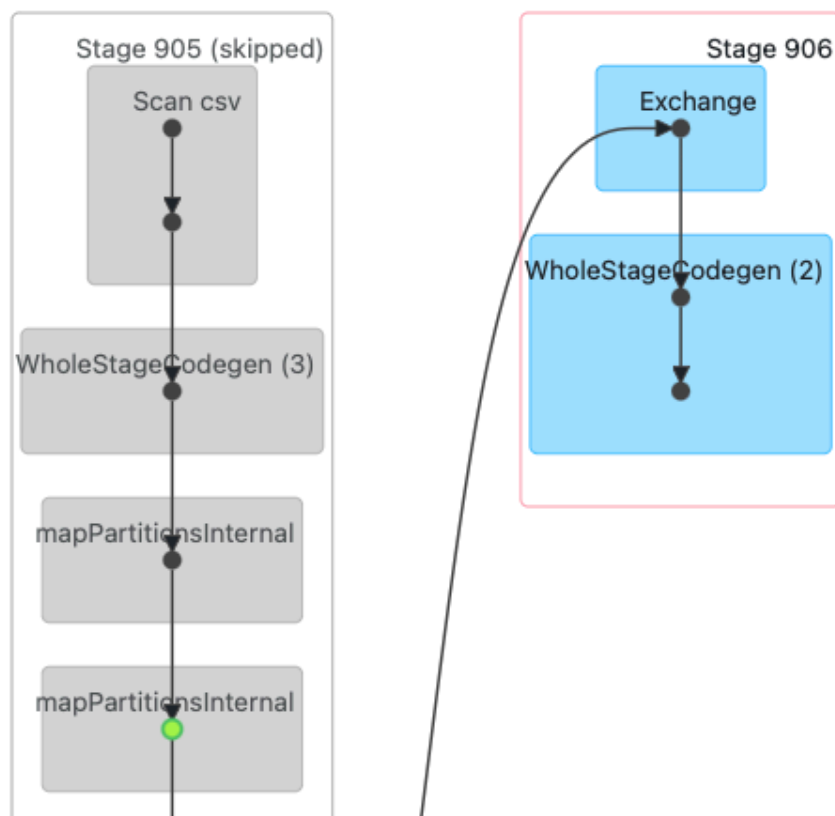
Associated SQL Query: 191

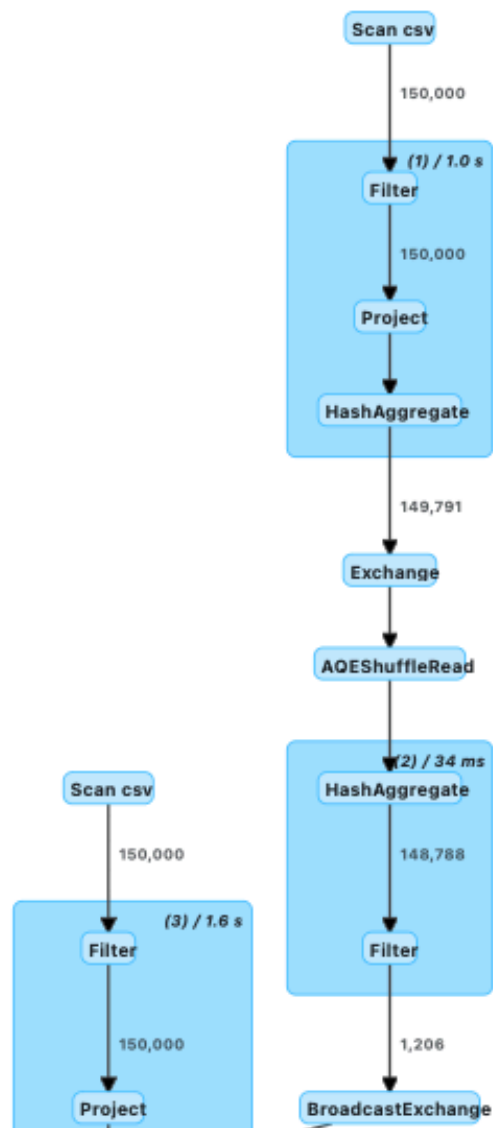
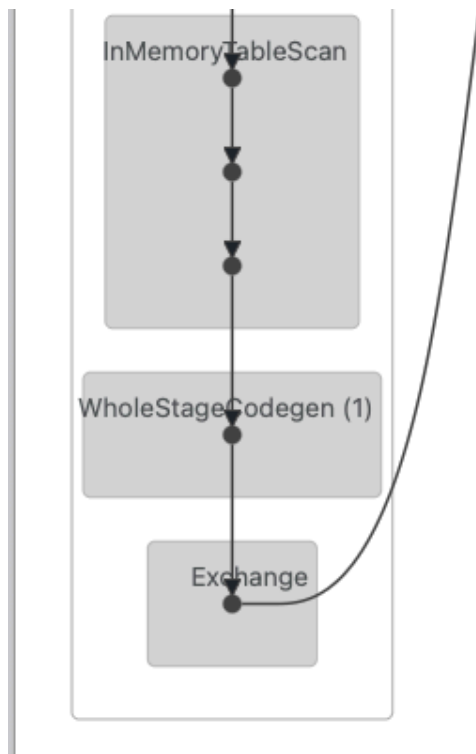
Completed Stages: 1

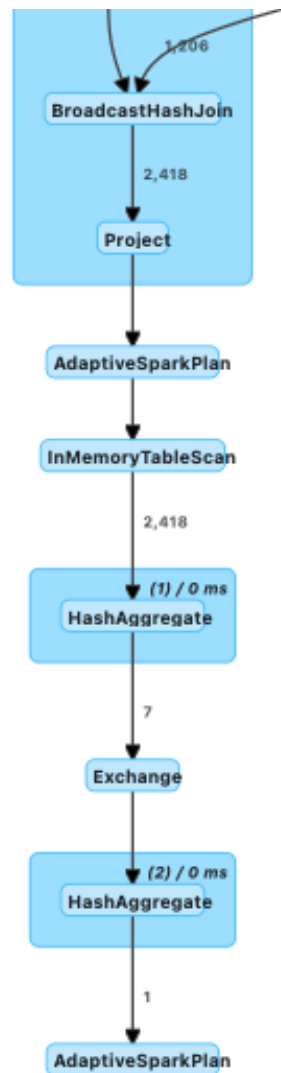
Skipped Stages: 1

[> Event Timeline](#)

[✓ DAG Visualization](#)







906	\$anonfun\$withThreadLocalCaptured\$1 at CompletableFuture.java:1768	+details	2026/09/20 12:26:17	6 ms	1/1		413.0 B	
904	\$anonfun\$withThreadLocalCaptured\$1 at CompletableFuture.java:1768	+details	2026/09/20 12:26:17	10 ms	7/7	312.0 KIB		413.0 B
903	count at <unknown>:0	+details	2026/09/20 12:26:17	0.1 s	7/7	24.7 MiB		
902	\$anonfun\$withThreadLocalCaptured\$1 at CompletableFuture.java:1768	+details	2026/09/20 12:26:16	37 ms	1/1		1486.5 KIB	
900	\$anonfun\$withThreadLocalCaptured\$1 at CompletableFuture.java:1768	+details	2026/09/20 12:26:16	0.2 s	7/7	24.7 MiB		1486.5 KIB

From the above screenshots, there are two stages involved with this query. Stage 905 is skipped, as it is being called from outputs completed previously in the code but the stage is separated by Exchange, shuffling the data between stages. However, if we look at the associated SQL query 191, there are five stages involved in this query with two main branches. Before the branches are joined, a lot of the operations are similar, Scan csv, filter and project, which means this would be optimised to reduce this computing cost. For example, if the original dataset was cached it would not have to be read in again each time it's called. This improvement can be seen from the timing of two initial blue boxes, with the left branch taking 1.6s and the right branch taking 1.0s. This computing time could be greatly reduced if this operation did not need to occur multiple times throughout the code and could be called whenever needed.

In the first branch, after the data is shuffled once removing null entries and

AQEShuffleRead reads the data back in. This is the final count, as discussed in Section 3, combining the data after partitioning when completing the GroupBy aggregation with the creation of repeat_bookings. At BroadcastExchange, the separation occurs to start the joining of the data with 1206 repeat Customer IDs found and at BroadcastHashJoin the data is joined with the associated 2418 rows that have the Booking IDs for these Customer IDs. Since repeat_booking_detail is cache to optimise memory storage and efficiency, additional stages are completed to add to memory with InMemoryTableScan and count() activating the cache. Regarding skew, as discussed previously, due to the smaller size of the dataset Spark assesses the degree of partitioning required and combines small partitions as required. Interesting with this dataset of 150,000 entries, all partitions are combined into a single partition. This is seen between Stage 900 and Stage 902, with skew seen in Stage 900 but in the next stage all data is within a single partition.

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	95.0 ms	0.2 s	0.2 s	0.2 s	0.2 s
GC Time	22.0 ms	22.0 ms	22.0 ms	22.0 ms	22.0 ms
Input Size / Records	362.5 KiB / 2180	4.1 MiB / 24613	4.1 MiB / 24644	4.1 MiB / 24650	4.1 MiB / 24650
Shuffle Write Size / Records	36.4 KiB / 2179	241.4 KiB / 24576	241.7 KiB / 24609	241.8 KiB / 24615	241.9 KiB / 24616
Task Deserialization Time	4.0 ms	4.0 ms	4.0 ms	4.0 ms	4.0 ms
Peak Execution Memory	2.2 MiB	2.2 MiB	2.2 MiB	2.2 MiB	2.2 MiB
Shuffle Write Time	29.0 ms	62.0 ms	65.0 ms	67.0 ms	67.0 ms

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	35.0 ms	35.0 ms	35.0 ms	35.0 ms	35.0 ms
GC Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Shuffle Read Size / Records	1.5 MiB / 149791	1.5 MiB / 149791	1.5 MiB / 149791	1.5 MiB / 149791	1.5 MiB / 149791
Task Deserialization Time	1.0 ms	1.0 ms	1.0 ms	1.0 ms	1.0 ms
Shuffle Read Fetch Wait Time	0.0 ms	0.0 ms	0.0 ms	0.0 ms	0.0 ms
Shuffle Remote Reads	0.0 B	0.0 B	0.0 B	0.0 B	0.0 B
Peak Execution Memory	18 MiB	18 MiB	18 MiB	18 MiB	18 MiB

Reviewing the ShuffleReadSize for Stage 900, there is skew with a minimum partition having only 2179 records while the median partition size has 24609 records. However, the record in the partitions is defined as too small and automatically combined into a single partition in the next stage, eliminating skew for this specific dataset. For a larger dataset, there would be more of an impact on computing time and overhead.